IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.0000000

Frontier and Localhost: A Survey of How Production AI Learns Outside the Weights

MIKHAIL L. ARBUZOV[1], LEE MOSBACKER[2], SISONG BEI[3], ZIWEI DONG[4], DMITRI KALAEV[5], AND ALEXEY SHVETS[6]

[1]Independent Researcher, CITY, COUNTRY (e-mail: mike.arbuzov54@gmail.com)
[2]Independent Researcher, CITY, COUNTRY (e-mail: lee.mosbacker@gmail.com)
[3]Independent Researcher, CITY, COUNTRY (e-mail: qurining@gmail.com)
[4]Independent Researcher, CITY, COUNTRY (e-mail: ziwei.dong@alumni.emory.edu)
[5]Independent Researcher, CITY, COUNTRY (e-mail: kalaevdr@gmail.com)
[6]Palo Alto Networks, CITY, COUNTRY (e-mail: ashvets@paloaltonetworks.com)

Corresponding author: Mikhail L. Arbuzov (e-mail: mike.arbuzov54@gmail.com).

TODO(authors): funding statement, e.g., "This work received no external funding." A. Shvets is employed by Palo Alto Networks; the views expressed are the authors' own.

ABSTRACT Every serious AI deployment now improves itself outside the weights. After a failure, a team edits instructions, skills, memories, tools, orchestration, or evaluation suites, and the system changes within the hour; the weights, on a cycle of months. This survey of 256 production and research systems asks what discipline that layer needs and finds most of its optimizer built, never assembled. The optimizer is one loop with two halves: the local loop traces a recurring failure to the artifact responsible, makes one edit, and keeps it only if the deployment's failure rate falls; the promotion loop decides how far it travels, to a team, an organization, or every customer. Nearly every failure-driven system closes the local loop by program but hardly builds its trace. The promotion loop exists in parts: no surveyed system assembles them into promotion across organizations with lineage, access control, and rollback, so every customer relearns the same fixes and nobody knows which rule helped. The reason is structural. What a deployment can automate is fixed by the feedback that keeps or rejects an edit: an exact check, an observed outcome, a judgment, or a reward months late. An accepted edit improves only what that feedback measured, so no lift travels on its own. The survey turns that finding into tools: five conditions that separate an optimizer from a pile, four feedback classes that say which half a deployment can automate, four properties to audit the promotion loop against, and the piece each class should build first.

INDEX TERMS AI governance, agent memory, agent skills, evaluation, large language models, LLM agents, prompt optimization, self-improving systems, survey

I. INTRODUCTION

A growing share of the adaptation that makes a deployed AI system reliable now happens in the layer of instructions, memories, skills, and tools around a frozen model, and this survey argues that the industry has built most of an optimizer for that layer without ever assembling it. Assembled, that optimizer would replace a pile of fixes that nobody can evaluate with a loop that keeps only what measurably helps.

The standard mental model of a system built on a large language model (LLM) is the model. Pre-training fixes the weights, fine-tuning nudges them, and deployment wraps a thin layer of prompts and tools around the result. Production in 2025 and 2026 has stopped behaving this way. Frontier weights change on a cycle of months; the systems built on them change daily, in instruction files checked into repositories, memories that persist across sessions, skills loaded on demand, registered tools, evaluation suites that block a merge, agent topologies, and pull-request gates on shared rule repositories. We call that fast-changing layer around a slow-changing model the scaffold.

Read one at a time, these artifacts look like unrelated engineering. A skill file is a procedure the model did not have; a repository rule is a convention for one project; a retrieval connector is memory access for one workflow; a tool registry provisions a capability the base model cannot reach; a pull-request review against a shared rule repository is a promotion gate. Read together, they are one adaptation surface, the non-model part of what Zaharia et al. call a compound AI system [1], and most deployments maintain it as patchwork. A failure happens, someone adds a rule, and the pile grows. Nobody

knows which addition helped, a fix validated in one place leaks into another that never tested it, and every customer relearns the same lesson.

The optimizer is one loop with two halves. Take a coding agent that keeps misnaming a class in one repository. The local loop traces that recurring failure to the rule file responsible, proposes one edit, and keeps it only if the repository's own failure rate falls. The promotion loop decides how far the edit may travel, to a team template, to the company's shared instructions, or to a registry shared across companies, and lets it travel only as far as evidence supports. Five conditions make the loop an optimizer rather than good intentions. The decisive one, a measurable loss, carries two consequences the paper returns to throughout: an accepted edit improves only what its gate measured, and the loop descends to that measure's ceiling rather than evolving toward some better goal.

Set side by side, the 256 systems in this survey tell one story about that loop. The local loop closes under a program nearly everywhere a failure-driven loop runs, in most research systems and a growing number of production ones. It closes from the gate backward: the gate is the best-built step, and the least built are the log of failures and the trace from a failure to the scaffold coordinate (a rule, skill, memory entry, or tool binding) that caused it. Around the gate the newer systems wait for several pieces of evidence before keeping an edit, remove stale skills through the same check that admitted them, and keep a history of every change to memory.

The promotion loop exists only in parts. Beyond the deployment, the newer systems have settled on one artifact format and one tool protocol, and in a few cases carry edits between deployments of one operator, the organization that runs both sides with no customer boundary between them. None of the 256 assembles those parts into promotion across organizations under versioned lineage, access control, and rollback. The reason is structural rather than a matter of effort. What a deployment can automate is fixed by the feedback its gate receives: a test or an observed outcome can gate an edit by program, a judgment only once the judge is calibrated against people, and a reward that arrives months later cannot gate edits one at a time. Because the lift an edit earns is bounded by what its gate measured, that lift reaches another deployment only if the receiver measures the same thing or measures again.

The rest of the survey turns that story into checks a team can run on its own system. It first states the loop in plain language and as an update rule, with the five conditions that tell a team whether its own maintenance is an optimizer or a pile (Section III). It then gives the rule that decides, from the feedback a deployment can get, which half of the loop it may automate today and which failure it should expect (Section IV). The corpus tests that rule: the field has converged on the local loop and sorted on the promotion loop as the rule predicts, with one qualification the data adds, and five research lines have arrived at the same loop from five different starting problems (Sections V to VIII). For the promotion loop, still unassembled, the survey ends with what any future system must show: four governance properties, the check without which no promotion is safe, the failures the literature has already measured in their absence, and, for each feedback class, the piece to build next (Sections IX to XI). The survey runs no experiments; its evidence is the published behavior of 256 systems, scored on a released table.

II. WHY ADAPTATION LIVES OUTSIDE THE WEIGHTS

Deployed systems adapt outside the weights because frontier training is built for breadth while a deployment's reliability is won in depth, in the conventions and failures particular to it. That mismatch is why a team's fixes land in the scaffold, and why this survey studies how those fixes are kept and how far they travel rather than how the weights are trained. Two earlier results in this series suggest why a gated scaffold is the right architecture and not a workaround, and the question this survey asks of it is one its neighbors leave open.

A. BREADTH IN THE WEIGHTS, DEPTH IN THE SCAFFOLD

Frontier training optimizes the weights for breadth, across every deployment at once, so the depth that makes one deployment reliable ends up in the scaffold.

The weights are tuned to generalize across millions of unseen deployments, and the objective smooths over exactly what one deployment cares about: local naming conventions, contradictory preferences between customers, rare workflow-specific failures. Because reinforcement learning from human feedback reduces output diversity and homogenizes preferred style across users [2], [3], and instruction-tuning data rewards patterns that generalize across the annotator pool, the frontier model is trained to disfavor one customer's idiosyncrasies. Tiwari et al. [4] reach the same place from the optimization side: in-context adaptation is cheap but capacity-bounded, in-weights adaptation is expressive but induces forgetting and loss of plasticity [5], so a system that learns slowly in the weights needs a cheap outer layer that learns fast outside them.

Encoding every deployment in the weights is ruled out three times over: fine-tuning per customer multiplies the training, evaluation, and serving lifecycle by the number of deployments; weights ship quarterly at best, while the deployment wants daily updates; and overfitting the weights to one customer degrades the neighbors whose preferences contradict. In principle, per-customer adapters [6], distilled models, and learned routing could absorb some of the residual, the failures that remain after training. In practice they are rare. Adaptation lands in the scaffold, the one substrate that is cheap, inspectable, reversible, scoped to one customer, and updatable daily.

B. WHY THE SCAFFOLD IS THE RIGHT ARCHITECTURE: THE TWO EARLIER RESULTS

Two earlier papers in this series offer a hypothesis that makes the gated scaffold the right architecture rather than a workaround, namely that a deployment's errors hinge on a

few key decisions that fail in a small, recurring set of ways. Without that hypothesis, the case for learning outside the weights would rest on cost alone.

Beyond Exponential Decay [7] argues that long outputs do not fail at the rate a uniform per-token error model predicts, because reliability hinges on a sparse set of key decisions. Its evidence is twofold: a direct measurement of long-range dependence on one natural-text corpus, where under a tenth of tokens depend on distant context, and the accuracy that majority voting recovers on reasoning benchmarks. The argument stops where the number of decisions grows with the task, as in long-horizon agentic work.

For a deployment that operates inside a bounded slice of recurring tasks, tools, and evaluator expectations, *The Architecture of Errors* [8] argues that those decisions fail in a small recurring catalogue of modes whose size grows only slowly with the number of observed failures. A targeted intervention budget then covers the residual; that budget grows polylogarithmically in sequence length and becomes constant once the catalogue saturates. The paper's own negative result marks the limit: no finite catalogue covers an unbounded domain.

If those results hold in general, a deployment's errors are few enough to fix one at a time, and a small catalogue of fixable modes is exactly what a cheap, inspectable, reversible artifact layer is for. This article asks where the fixes live, and the corpus answers: in persistent artifacts outside the weights, one recurring mode at a time, which is the pattern the hypothesis predicts. On that hypothesis the gated and scoped layer is not a workaround for an inadequate model. It is the architecture the error structure calls for: a frontier model that generalizes, a local scaffold that specializes, and reliability that comes from governing the specialization. The survey's own findings, in Sections VI to IX, stand without it.

C. RELATION TO EXISTING SURVEYS

No neighboring survey asks the question this one is organized around, which is who decides that an edit is good and how far the edit may travel. It is the question a team faces each time it decides whether to keep a fix and whether to share it.

The neighboring surveys explain how agents evolve, what their memories store, how context is engineered, how skills are evaluated, and how harnesses are built. The question asked here has four parts: whether a feedback loop closes at all, whether its gate is a program or a person, what the gate can observe, and whether promotion beyond one deployment is governed. Table 1 says, for each neighbor, what it classifies by and where this survey departs from it.

On the governance part, two neighbors come close. Ding et al.'s survey of persistent-state agents finds the literature concentrated on accumulating and retrieving state rather than on governing, recovering, or relinquishing it [9], and the memory-security survey of Lin et al. scopes and rolls back memory for safety [10]; neither asks who gates an edit, and neither covers promotion across organizations.

A reader will also expect the prompt-optimization lineage: OPRO, EvoPrompt, Promptbreeder, DSPy, and TextGrad [18]–[22]. That lineage optimizes a prompt or program once against a development set and is not a deployed loop; agentic context engineering [23] does persist its artifact, an evolving playbook, but never promotes it. The closest precedents optimize the scaffold itself under a benchmark score: agent symbolic learning updates prompts, tools, and pipeline as symbolic parameters from textual feedback [24], STOP's scaffolding program improves its own improvement code [25], and a coding agent edits its own harness to raise its score [26]. What changes here is that the edited artifact persists in a deployment and must clear a second gate before it earns any scope beyond it (Supplementary Material S7).

III. THE LOOP: ARTIFACT-LAYER DESCENT

Scaffold maintenance becomes an optimizer, which this survey calls artifact-layer descent, when every fix is tested against a measured failure rate before it is kept and again before it travels. That definition gives a team a test it can apply to its own maintenance, and it gives the survey the yardstick it holds every system in the corpus against. Below, the loop appears in plain words, as five conditions with an update rule, and in one coding agent's path from patchwork to descent.

A. THE LOOP IN PLAIN LANGUAGE

Artifact-layer descent is one loop with two halves, run again and again over the life of a deployment. The local loop fixes a recurring failure inside one deployment; the promotion loop decides how far the fix may travel. (Figures and the released table label them Loop 1 and Loop 2.)

Fig. 1 draws the local loop. The deployment runs and logs what goes wrong: failed outputs, user corrections, rejected answers, evaluation regressions, tool-call errors. The loop does not react to single failures. It clusters the log into recurring modes and takes up the ones that matter most, ranked by how often each recurs and how badly it fails. For such a mode it locates the scaffold coordinate responsible, which may be a stale memory entry, an under-specified instruction, a missing tool argument, a brittle retrieval rule, or a misrouted agent edge, and it proposes one candidate edit to that coordinate.

Then it measures. A deployment serves one **patch**: the slice of users, tasks, and conventions it handles. The failure rate on that patch's recurring tasks is the **patch loss**. The local gate accepts the edit if the patch loss falls and rejects it otherwise. An accepted edit stays in the scaffold, versioned; one that a later check shows made things worse is rolled back. That is the whole of the local loop, and most research systems and a growing number of production systems now run it, though rarely every step of it (Section VI).

The promotion loop is the half most systems skip. An engineer notices that a coding agent keeps misnaming a class in one repository. The fix is a one-line rule in that repository's instruction file. That same rule could be promoted to a team-wide template, to the company's shared instructions, or to a skill registry shared across companies. The edit did not change; its scope did. And the gate that should let it through gets stricter at each step, because the loss it is tested against is

TABLE 1. Neighboring surveys and where this one departs from each. This survey classifies by loop closure, gate type, signal class, and governance, and is the only one to cover promotion across organizations.

Survey	Scope	How this survey differs
Gao et al. [11]	self-evolving agents	organizes by what, when, how, and where an agent evolves; the gate that accepts an edit, and the edit's scope, are mentioned but not classified
Fang et al. [12]	self-evolving agents	organizes by inputs, agent system, environment, and optimizer; loop closure appears in passing, gates and promotion do not
Zhou et al. [13]	self-evolving coding agents (108 methods)	organizes by evolution target, timing, and code signal; closest on reversibility, with no gate or promotion axis
Ding et al. [9]	persistent-state agents (435 works)	organizes by authority, scope, mutability, provenance, and recoverability of state; governance is an axis, the gate on an edit is not
Lin et al. [10]	memory security (~90 works)	organizes by lifecycle phase and security objective; principal scoping and rollback are axes, feedback and promotion are not
Guo et al. [14]	agent system and harness design	organizes by six runtime responsibilities; touches all four questions without classifying by any
Xia et al. [15]	skill-mediated agents (8 systems)	organizes by four responsibility layers and ten patterns; loop closure and governance appear as patterns, not as axes
Ding et al. [16]	skill evaluation and evolution	organizes by four evolution paradigms and six benchmark classes; evaluation is the subject, the gate it feeds is not
Mei et al. [17]	context engineering (1,411 papers)	organizes by components and system implementations; adaptation loops lie outside its scope

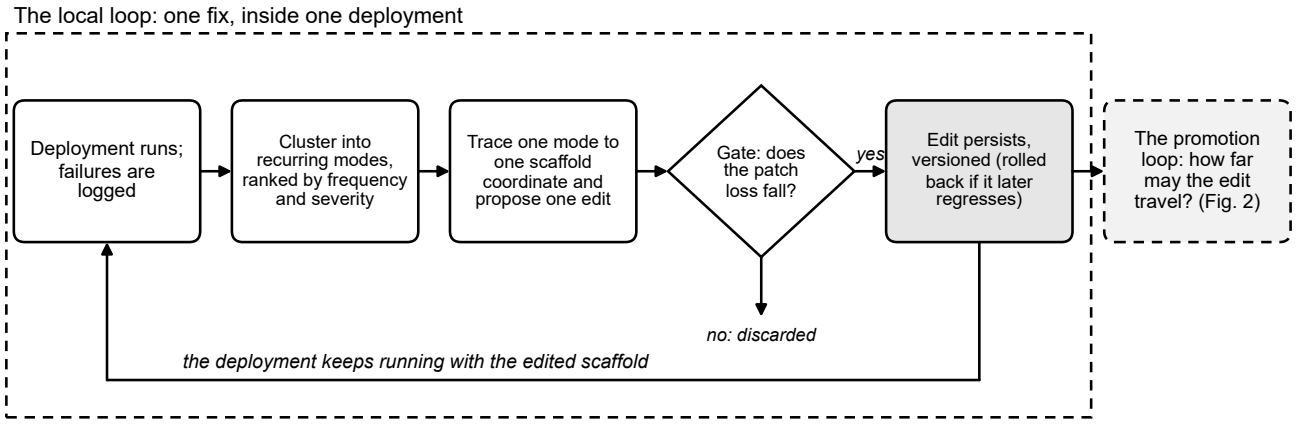


FIGURE 1. The local loop of artifact-layer descent. A recurring failure is traced to one scaffold coordinate, one edit is proposed, and a gate keeps the edit only if the deployment's patch loss falls. An accepted edit then enters the promotion loop (Fig. 2).

now an average over more deployments, and a fix that helps one repository can hurt another.

The scopes nest (Fig. 2). USER is one person's account or fork; PROJECT is one repository, workflow, or product workspace; STACK is a curated bundle of projects under one team; a **tenant** is one organization or customer; CORE is the scope every tenant shares. Each scope's loss is the average failure rate over the deployments it contains. Promotion that reaches CORE crosses tenants, which is what cross-tenant promotion means here. The promotion loop moves an edit upward only as far as the evidence reaches; we call that limit the edit's **evidence radius**. The check that belongs at each rung is **receiving-side evaluation**: a check run where the edit arrives, not where it was made. In optimizer terms, the local loop sets the step (how much one accepted edit changes the local scaffold) and the promotion loop sets the target (which average failure rate the scaffold is tuned to reduce). Governance is therefore not administrative overhead. It is the regularization that keeps a small fix from being aimed at the wrong target.

B. WHEN THE LOOP IS AN OPTIMIZER

The loop is an optimizer, and not a description of good practice, only when five conditions hold, the last of them a gate that decides which edits are kept.

The five conditions are these:

1. a measurable patch loss, such as a failure rate, a correction rate, or an evaluation score;
2. a failure log, so that residuals are captured rather than lost;

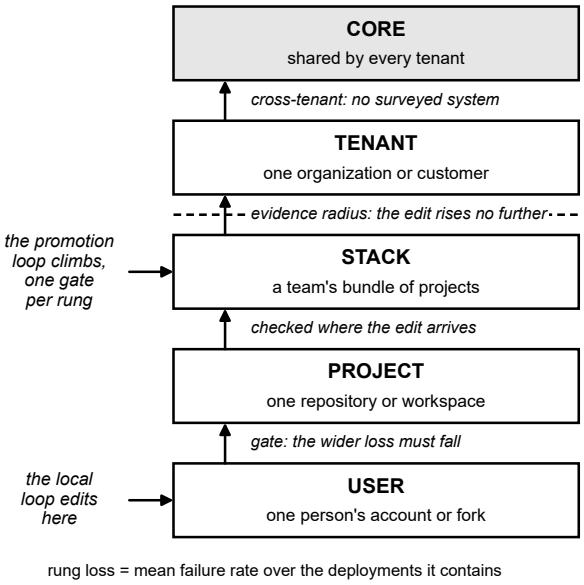


FIGURE 2. The promotion loop. Scopes nest from USER to CORE, each rung's loss averages the deployments it contains, and an edit rises one rung at a time, only while a check at the receiving side shows the wider loss falling and only as far as its evidence radius. Promotion into CORE crosses tenants.

3. credit assignment, so that a failure can be traced, at least sometimes, to the scaffold coordinate that caused it;
4. a synthesized candidate edit to that coordinate; and
5. a gate that checks whether the edit lowers the loss before the edit is kept or promoted.

The conditions presuppose frozen frontier weights, edits that persist as inspectable artifacts rather than transient prompts, and promotion that widens an artifact's scope under a gate rather than averaging comparable updates as weight-space federated learning does.

Written out, the patch loss of deployment D is

$$L_D(\theta_s) = \mathbb{E}_{x\sim D}[\ell(\pi(\theta_M, \theta_s, x))]\,,$$

where θ_M are the frontier weights, fixed during deployment, θ_s is the scaffold, a structured object over the six substrate coordinates of Section V, and ℓ is an operational loss. A failure yields no analytic gradient. It yields a candidate edit z_t, proposed by reflecting over the accumulated failure history. The gate accepts the edit when its noisy estimate of the change in loss clears a margin,

$$G_t(z_t) = 1 \iff \widehat{\Delta}_{z_t} L_D(\theta_s^t) + \lambda\,\Omega(z_t) \le -\tau_t,$$

where $\Omega \ge 0$ penalizes the complexity of the edit and the breadth of its promotion, and $\tau_t \ge 0$ is the required margin. Production systems implement this inequality without ever writing it down: an evaluation blocks a merge on regression, a reviewer rejects a vague rule, a governance system holds an artifact back until evidence accumulates. The update satisfies a one-step descent inequality under a bounded-bias assumption on the gate. When that bias is summable over the run, the loss converges and improving edits stop clearing the gate; the scaffold is then gate-stable, which is a statement about the gate, not about optimality (Appendix XII). Where a condition fails, the picture is an analogy, and the survey marks which systems meet which conditions.

Each half of the loop has a gate, the local gate and the promotion gate, and we name three kinds by who decides. A gate is **Auto** when a program applies a check before the edit goes through: an evaluation score, a verifier, a reinforcement-learning threshold, or a language-model judge [27], whether or not its agreement with people has been measured, a question Section IV returns to. It is **Human** when a person decides. It is **None** when nothing decides, either because that half of the loop is absent or because the step runs unchecked, as when a system pushes every edit to its neighbors without a test. Who writes the candidate is recorded separately and is not a gate: the author may be an agent reflecting over failures, an engineer at a desk, or a contributor registering an artifact. We write a system's two gates as a pair, the local gate first: [Auto × Human] means an algorithmic local gate and a human promotion gate. These pairs index the design space of Section VII.

The first condition says the loss must be measurable. It does not say what measures it, and that turns out to be the question that sorts the corpus.

C. FROM PATCHWORK TO DESCENT

Scaffold maintenance passes through three stages that recur across the corpus (patchwork, random artifact search, and artifact-layer descent), and the five conditions are what separate them. The stages matter because a loop that reaches descent stops failing like a pile of fixes and starts failing through its gate.

Take the coding agent above that keeps misnaming a class. At the first stage, patchwork, an engineer adds a rule to the instruction file after noticing the failure; nobody measures whether it helped, nothing is promoted on evidence, and the file sprawls with every incident until nobody knows which rule does what. At the second stage, random artifact search, the system tries many variants of the file under the test suite and keeps the best. The suite is the only signal and no failure is traced to the rule that caused it; the winning variant has learned the suite as much as the task. At the third stage, artifact-layer descent, the recurring failure is traced to the one rule responsible, one edit is proposed, the gate checks that the patch loss falls, and promotion stops at the evidence radius and can be reversed.

At the third stage, the suite or judge that gates the edits is itself a scaffold artifact that drifts and can be gamed. Which failure dominates depends on the signal the gate receives (Sections IV and X). Production deployments that reach the third stage in the local loop then promote in one of two ways: behind a human gate, as Spotify does [28], or, in Letta's Context Repositories alone, by an ungated push that carries lineage but no evaluation.

A measurable loss has two consequences that follow the loop everywhere. The first is that the measure bounds the lift. An accepted edit is an improvement in what the gate measured, on the tasks the gate sampled, and nothing more; a rule that lowers the failure rate on one repository's suite has earned nothing about another repository's. Every loss has a scope. That is why the promotion loop cannot carry the local loop's evidence: the receiving scope measures a different mix of tasks, so a promoted edit either travels under a measure the scopes share or re-establishes its lift where it arrives, which is what receiving-side evaluation is for. Where tenants measure different things, a lift does not transfer at all without it.

The second is that the loop optimizes and does not evolve. It needs a defined measure before it can start and descends toward that measure's ceiling; at the ceiling no edit clears the gate, which is what gate stability means. It does not discover a better goal than the one its measure encodes. Two scaffolds optimized for the same goal under the same measure need not converge to one version; each stops at the ceiling by its own route. A system that edits its own measure, the reflexive gate of Section IV, moves the destination rather than evolving toward an unknown truth. The distinction matters because the corpus is often described in the vocabulary of evolution. What the gated systems in it actually do is descent.

IV. WHERE THE SIGNAL COMES FROM: FOUR CLASSES OF FEEDBACK

What a deployment can automate is fixed by the feedback its gate receives, which is either an exact check, an observed outcome, a judgment, or a reward that arrives months later. A team that knows which of the four it has knows which half of the loop it can hand to a program today and which failure to expect, and the same rule predicts how the surveyed systems divide.

Four scaffolds all run the loop of Section III. A coding agent changes one line in its repository rule file; the test suite runs and reports pass or fail within seconds. A scheduling assistant changes how it phrases a booking request; the appointment either appears on the calendar or it does not, and the system learns which an hour later. When a marketing assistant changes the tone rule in its instructions, nobody can say whether the next draft is better, only whether a reviewer prefers it. A research assistant changes how it chooses which direction to pursue; whether the choice was good becomes visible months later, after many other edits have landed. All four produce a candidate edit. They differ in what the gate can know about it.

The gate needs an estimate of one quantity: whether the edit lowered the patch loss. That estimate comes from a feedback signal, and signals differ in three ways: how soon they arrive, how exact they are, and whether there is a fact of the matter at all. Those three differences sort every surveyed system into four classes (Table 2), and the class fixes what the gate can be.

The classes are a partition, not three independent axes: a system is assigned by the property that binds its gate first. Delay binds before anything else, so an outcome that arrives months later is C4 even when it is objective. The absence of ground truth binds next, so an immediate judgment is C3. Among signals that are prompt and objective, exactness separates C1 from C2.

C1, exact. A deterministic checker returns pass or fail, or an exact score, immediately and repeatably: unit tests, compilers, type checkers, exact-match answer keys, proof checkers, schema validators. DGM runs every self-modification in a sandbox and keeps every one that still compiles and can edit code, a functionality check on ten benchmark tasks; the benchmark score then weights which archive members are chosen as parents rather than deciding which are kept [29]. AlphaEvolve scores candidate programs with a cascade of exact evaluators [30], and SkillOpt gates a skill edit on a held-out exact-match split of a math benchmark [31]. Because the check is exact for the artifact and the test it runs, and repeating it returns the same answer, one run settles the checked case. What the check does not settle is the patch loss: the suite is a sample of the patch's task mix, and the agent that runs under the edited scaffold is stochastic, so the gate's estimate of the loss change still carries sampling error. The gate can be a program and its margin can be small, but the margin absorbs sampling error, not error in the check. The improvement the check certifies is also bounded and specific: an improvement in what the check measures, on the cases it ran, not an improvement in general. Two things stay hard. A failing test localizes the output, not the scaffold coordinate that caused it, so credit assignment remains open; and a checker that is also the training target can be gamed. The dominant failures are scaffold bloat, because cheap acceptances accumulate, and overfitting to the suite, which Section X files under evaluation fragility.

C2, observable outcome. The environment or the user emits an outcome after an episode: a booked appointment, a closed ticket, a merged pull request, a tool call that returns without error, a retrieved document that contains the answer. The signal is real but one episode late, noisy, and confounded, since a user can accept a wrong answer and reject a right one. An outcome from a simulator such as Voyager's game world or SAGE's AppWorld is the same kind of signal without the confound, because no user is in the episode and it can be re-run until the noise averages out; it is what Section VII calls a simulated outcome. Gates of this class include Devin's merged pull requests [32], SAGE's task success on AppWorld [33], and PACE's failure signals from a deployed small model [34]; SkillRL reads the same signal only as a trigger, a success rate that decides when new skills are proposed but not whether they stay [35]. The gate can still be a program (a reinforcement-learning threshold or a held-out task suite), but it needs a positive margin, and it absorbs the noise by batching episodes or by requiring agreement across samples (Appendix XIII). Credit assignment takes its most stubborn form here: the outcome says *that* the episode failed, not *why*. Most production deployments live in this class, and its signal arrives through a channel an attacker can control, which is why poisoned memory and tool-output injection dominate.

TABLE 2. The four feedback-signal classes. Reading across a row: what the gate receives, what it can therefore be, and what it cannot fix.

Class	The signal the gate receives	What the gate can be	What stays hard	Corpus examples
C1 exact	A check that returns pass or fail, or an exact score, at once and repeatably: tests, compilers, answer keys	An automatic check; one run per case, repeated only to sample the task mix	The lift is bounded and specific, an improvement in what the check measures on the cases it ran; the check names the failed output, not the coordinate that caused it; a check that is also the target can be gamed	DGM, AlphaEvolve, SkillOpt
C2 observable outcome	A real result one episode later: a booked slot, a merged pull request, a closed ticket; noisy and confounded	An automatic threshold over batched episodes; a person at promotion	The outcome says *that* the episode failed, not *why*; the channel can be poisoned	Devin, Sierra, SAGE, SkillRL, PACE, Voyager
C3 judged	A rater, a rubric, or a model judge; no ground truth	A judge calibrated against people; a person at promotion	The rubric is itself a scaffold coordinate, so the gate judges edits to itself; the proxy gets optimized past the point where it tracks quality	SkillForge, CASCADE, skills-repository review
C4 delayed	An outcome weeks or months later, once per many edits	No per-edit gate; proposals are pruned and milestones stand in	Credit across many edits; drift; the loop cannot be told from random search	NanoResearch, CORAL, the AI Scientist

C3, judged. There is no fact of the matter. An essay, a marketing page, a design, a pitch deck, or a code review for style can be scored only by a judge: a human rater, a language-model judge, a rubric, or a comparison against references. In SkillForge a judge that agrees with people more than nine times in ten flags the failed replies that drive each skill rewrite, and the rewritten skill is committed without an acceptance test [36]; CASCADE consolidates memory under a human-and-agent gate [37]; Anthropic's skills repository promotes a skill when a reviewer approves the pull request. The gate can be a program only after the judge has been calibrated against people, because the judge's bias is now the estimator's bias. Residuals, meanwhile, are captured only when someone objects, so the failure log depends on complaint channels rather than instrumentation. The dominant failure is Goodhart's law in the form measured for reward models: optimizing a proxy past a point lowers the true score [38]. Once the judge is the target, the scaffold learns the judge. Nor is a human judge stable: Shankar et al. find that evaluators' criteria drift as they grade more outputs [39].

C4, delayed and sparse. The outcome arrives after weeks or months, once per many edits, in a world that has moved on. Research programs, product strategy, and open-ended discovery are of this kind; the clearest instances are automated research pipelines such as the AI Scientist, which generates ideas, runs experiments, and writes the paper [40]. NanoResearch merges candidate skills by semantic compatibility rather than by a measured loss change [41], and CORAL promotes findings into shared memory on a heartbeat rather than on an evaluation [42]. AlphaEvolve's evolutionary database can look like an instance, and is not one: it balances improving the best programs against diversity only after every candidate has been scored by its exact evaluators, so it is a search strategy inside the exact class, not a gate for a delayed signal [30]. With no per-edit estimate to gate on, the gate degenerates to proposal pruning plus proxy milestones, and the descent argument covers only the composition of pruning and whatever gate exists at the horizon (Appendix XIII). Temporal credit assignment across many edits is open. Drift dominates: the scaffold ages faster than the signal that would correct it. An optimizer in this class also cannot be told from random search by its outcomes alone; Ong et al. [43] report the same for harness optimizers.

The judge is part of the scaffold. The judged class has one feature the other three lack: the rubric that sets the loss is itself a scaffold coordinate, and it returns in the findings and in the open problems. Suppose a pitch-review system keeps overrating decks that claim a large market without numbers. It traces the failure to its rubric and edits ''evaluate market size'' into ''require a numeric estimate and a source''. That edit changes the gate that will judge the next edit. We call this the *reflexive gate*, and nothing in the corpus calibrates one: three scaffold systems report judge calibration [36], [44], [45], and none calibrates a judge that is itself being edited.

The weight side faces the same question. Reinforcement learning from verifiable rewards lies outside the scored corpus, but pushing it into writing and other open-ended tasks has forced researchers to build judged gates. One group holds the gate fixed while the policy learns, as a checklist, a pairwise judge, a reference-anchored judge, or a fixed rubric [46]–[49]. A second group lets the gate move with the policy, scoring by the policy's own probability of a reference or updating the rubric or judge during training [50]–[54]. Fixed versus co-evolved is exactly the reflexive-gate question, and of the co-evolving methods only RLPR, whose reward is a probability rather than a rubric, reports agreement with human labels

(Supplementary Material S6).

What the class decides. Read down the table and three consequences follow, each a prediction about the corpus that Sections VI, VII, and X check. The first is which half of the loop can close automatically. An exact check or an observed outcome can gate a local edit by program. A judgment can do so only once the judge is calibrated, and a delayed reward cannot gate edits one at a time. Promotion is stricter. Only the exact class carries its own measure to the receiver, and only when the receiving side runs the same check: that is the one case in which the sending side's evidence means the same thing on arrival. In every other case the receiving-side evaluation of Section III is still owed. The second is which gate pairings a deployment can reach. [Auto × Auto] belongs to exact signals and simulated outcomes; [Auto × Human] belongs to observed outcomes and judgments, where the person at the promotion gate does the work the missing exact signal cannot; and the delayed class has no per-edit gate on either half. The third is which failure dominates: the table's column on what stays hard records it, and Section X develops it.

V. THE CORPUS, AND HOW IT WAS READ AGAINST THE LOOP

The survey's evidence is a corpus of public systems, each placed on the loop by reading its own published text for which part of the scaffold it edits, whether a measure gates the edit, who decides, and how far the edit travels. Every finding that follows is a count over these placements, so the reading is released row by row with a one-line justification, and any placement can be checked against the system itself rather than against this survey.

A. HOW THE CORPUS WAS ASSEMBLED

The corpus starts from public systems that adapt a foundation model through a persistent artifact or are described as self-improving, and two filters then narrow it, first to systems whose artifacts persist across scope or are updated from feedback, then to systems that both edit an artifact from session signal and promote it beyond the deployment that made it. Both filters are stricter than the field's own labels, so a system is placed by what it does rather than by what it is called.

A system enters the corpus if it wraps a foundation model with at least one persistent artifact updated from deployment signal, or if it is described in public as self-improving, scaffolded, or agentic; the corpus excludes systems whose only update mechanism is weight-level training and systems with no public evidence of deployment or release (Supplementary Material S2). The systems were found in public sources in two passes (130 by May 2026 and 126 more by the October 2026 cutoff). For each system it added, the second pass records the gate on each half of the loop, the signal class, and the governance properties as columns of the released table, so the counts by gate that follow cover the systems it added.

The first filter (Fig. 3) admits a system to the core corpus at a score of at least 3 on at least one substrate on the ruler below.

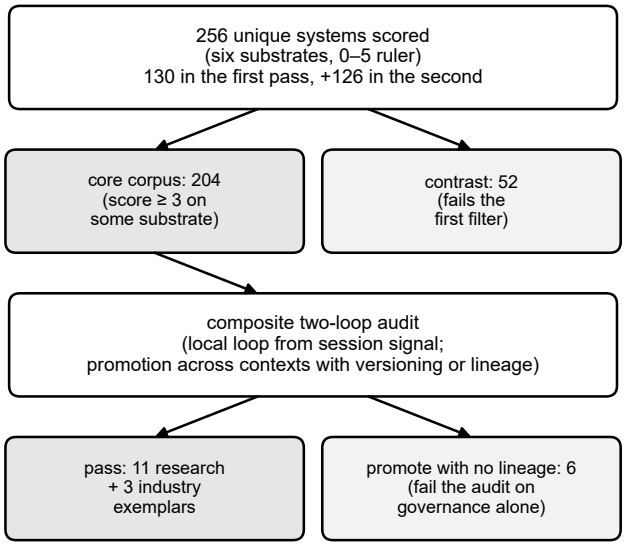


FIGURE 3. Corpus flow. Every system is scored on the substrates it adapts, with an integrated row for systems that adapt several, giving 269 substrate-level rows for 256 systems; the first filter splits core from contrast; the composite two-loop audit selects the systems placed in the design space of Section VII.

Systems that carry the field's labels but fail this filter form a contrast corpus, kept so that the boundary can be audited rather than asserted. Of the 256 systems, 204 form the core corpus and 52 the contrast corpus. Each system is scored on the substrate it adapts or, if it adapts several, in one integrated row, and a few are scored on two rows, which is why 256 systems give 269 rows.

The second filter is the two-loop audit, which asks each core system one question for each half of the loop: does the local loop modify a persisted artifact from session signal, and is that artifact then promoted beyond the deployment that produced it, with versioning or lineage? The audit was applied to the core systems and to the ten research systems classified from their papers alone (they are not rows of the released table), with eight exceptions (Supplementary Material S4). It admits 11 research systems (NanoResearch, SkillOpt, AlphaEvolve, ADAS, DGM, Voyager, SAGE, EvolveR, CoEvoSkills, AutoManual, and CASCADE; all but SkillOpt are rows of the released table) and 3 industry exemplars (Sierra Agent OS 2.0, Cognition Devin, and Letta's Context Repositories). Section VII places them, with the other audited systems and the contrast corpus, on the gate grid by who decides on each half. Passing the audit is a weaker thing than having the governed promotion loop of the thesis. Every passing system promotes inside one operator, either from one agent, user, or repository to the next or into a shared bank or archive that later runs draw from; none reaches another organization; and one of them, Letta, promotes with no gate at all (Section VII).

B. SIX COORDINATES AND ONE RULER

An edit can land on one of six substrates of the scaffold (instructions, skills, memory, tools, orchestration, and governance), and every system is scored on the substrates it adapts with one ruler, from 0 to 5, that measures how far along the loop the system has come on that substrate. The substrates say where the trace from a failure can point, and the ruler turns a system's text into the score the later counts are built from.

The six substrates (Table 3) are the coordinates of θ_s, each editable independently of the others. The three kinds of memory on the memory substrate follow the cognitive-architecture taxonomy of Sumers et al. [55].

At 0 the artifact is ephemeral; at 1 it is persistent and local; at 2 it is persistent with tool attachment but no feedback. Up to that point an artifact persists and nothing measures it, which is the patchwork stage of Section III. At 3 the artifact either spans scopes with no gate on the way or is updated from feedback by a person; at 4 a program updates it from feedback, so the local loop closes. The top mark, 5, differs by substrate (Table 3). On instructions and skills (S1, S2) it requires both halves of the loop; on orchestration (S5) it requires a topology that a loop outside the agent's own run updates from deployment feedback; and on the substrates that hold governance (S3, S4, S6) it requires the machinery the promotion loop would need, which is why a registry or a prompt platform can score 5 with no local loop at all. The integrated row scores breadth and depth of adaptation, not the loop, so its top mark says nothing about either half.

C. WHAT WAS GATHERED, BY SUBSTRATE

No system in the corpus matures all six substrates at once, because each kind of system was built to satisfy one constraint and matured only the substrates that constraint touches.

Research systems were built for fast learning: they learn fast and lack enterprise governance. Production systems were built for safe change: they govern well and learn slowly. Open standards were built for portability: they solve artifact portability but not adaptation. The top marks fall accordingly (Table 4): research systems on skills and memory and in the integrated row, products and standards on tools and governance, and no system at all on instructions or orchestration.

D. EVIDENCE AND SCORING DISCIPLINE

Every classification rests on public evidence of uneven strength and on one rater, and the scoring rules say how each kind of evidence counts and how close calls were settled. A reader weighing any count needs both.

Every placement in Sections VI to X is a reading, in the vocabulary of Section III, of one system's own method section or documentation. Each system carries one or more evidence tiers (T1 to T5 in the released table: peer-reviewed publication, arXiv preprint, production claim with metrics, open-source artifact, industry blog or interview). Peer-reviewed papers and open-source artifacts weigh most in corpus-level statements. A production claim counts when it names a benchmark or integration target, a blog post or interview counts only when a higher tier corroborates it, and production claims are never pooled with peer-reviewed evidence in a corpus-level statistic. Industry systems evaluated from vendor documentation rather than a paper are flagged wherever they appear.

One author scored every system, used half-scores only when one criterion was clearly met and another partially, and rounded ambiguous cases down. The scores have not been independently re-rated, and that check is future work.

The complete score table, with a one-line justification per row, is released so that every scored classification can be re-derived (Supplementary Material S1). Under natural relaxations of the audit (counting rollback alone as promotion governance, asking for a promotion mechanism with no governance at all, admitting a local loop that persists only optimizer state), the counts move while the qualitative findings survive every one of them (Supplementary Material S2).

VI. THE LOCAL LOOP IN THE INDUSTRY: A FAILURE, ONE EDIT, A GATE

The industry has built the local loop nearly everywhere a failure-driven loop runs, and it has built it from the gate backward. That order matters to anyone maintaining a scaffold, because a gate can only keep or discard the edit it is handed, and the steps that hand it one are the steps least built. The section follows the local loop in the order it runs, from the logged failure to one system that has all five conditions at once.

In more than four of five of the most recently added core systems a program keeps or discards each local edit on a measured loss, and the pattern holds on every substrate that has a failure-driven loop at all (Table 4). What stands before the gate is far less built: production deployments log preferences and successes rather than failures, so there is little to cluster, and no general method traces a failed output to the coordinate that caused it. What stands after the gate is half built: the memory products keep every entry's history and still return facts that were revoked.

A. THE FAILURE AND ITS TRACE

The two steps that feed the local gate, logging a failure and tracing it to the artifact that caused it, are the least built steps of the loop. They decide what the gate is ever asked to judge, so a loop that skips them is guessing which artifact to change.

A failure log means that failures are recorded rather than lost, and in production they are mostly lost. Only one field report describes a store that records dead ends so they are not repeated, and it covers a single, self-reported deployment [56]; everywhere else the log that the local loop would cluster is never written. The gap sits in the observable-outcome class (C2), where an episode is recorded as a success or a preference and the failure that would have been the residual leaves no entry.

Which sessions inform the next edit is left to chance. The loop of Section III ranks recurring modes by how often

TABLE 3. The six scaffold substrates, what each holds, and what a system must do on it to score 5.

Substrate	What persists	A score of 5 requires
S1 Instructions	Project, team, or organization rules attached to a workspace	Versioned rules promoted under an automatic gate
S2 Skills	Procedures, as code or text, loaded on demand	A skill bank that edits itself on failure, test-gated, versioned, and promoted
S3 Memory	Experience kept across sessions: episodic, semantic, procedural	Provenance, versions, and a correction path for every entry
S4 Tools	The vetted action surface and its schemas	Evaluation-gated tool versions; a curated bundle as the product
S5 Orchestration	The multi-role graph and its routing	Topology updated from deployment feedback by a loop outside the agent's own run
S6 Governance	Versioning, promotion, audit, rollback	Staged promotion with evaluation thresholds, an audit log, and rollback

TABLE 4. Substrate maturity across the 256-system corpus. Column n counts scored rows on each substrate, Mean is the average score, and Score-5 systems reach the substrate's top mark; the integrated row counts systems scored across several substrates as one row. Of the 126 most recently added systems, 105 are core and 32 are skill systems; 89 of those core systems close the local loop under a program, and none of the 126 scores 5 on any substrate.

Substrate	*n*	Mean	Score-5 systems	What the corpus holds on it
S1 Instructions	15	3.07	(none)	Rules persist across IDEs and agents and nothing measures them; no system closes both loops; ceiling 4
S2 Skills	50	3.34	SAGE, COSPLAY	The local loop closes under a program in the research systems; the production repositories merge by hand
S3 Memory	50	3.06	Cuadros et al.	The ledger ships in products; the correction a gate would enforce is absent
S4 Tools	20	3.65	MCP, Harvey, Hippocratic AI	The most mature production substrate; registries govern admission, not evaluation
S5 Orchestration	37	3.08	(none)	Topologies change under an evaluation gate; one structurally gated instance; no two-loop system with lineage
S6 Governance	45	3.02	Braintrust, Vellum, LangSmith Hub, AGENTS.md under the Agentic AI Foundation (AAIF)	Prompts-as-code mature; registries add lineage and rollback; no local loop generates a candidate
Integrated	52	3.81	NanoResearch, AutoAgent, SkillRL	Breadth and depth of adaptation, not the loop; reached only by research systems without enterprise governance

they recur and how badly they fail before it takes one up; nearly every system in the corpus takes failures in the order they arrive instead. Task-CoEvolve's variance-weighted task sampling and AutoSaddler's mini-batch failure signals are the exceptions [57], [58], and both weight the sessions the loop learns from by how much they disagree (C2). A loop that learns from whatever arrived last is descending on a sample nobody chose.

The trace from a failure to the coordinate that caused it, the credit assignment of the loop, is the step the field itself names as the bottleneck. The harness-optimization line [59]–[63] and the evaluation of harness optimizers by Ong et al. [43] agree that attributing a system-level failure to one scaffold coordinate is what separates descent from trial and error.

Where the trace is solved, it is solved one substrate at a time. Meta-Harness keeps a filesystem of candidate harnesses that records which edit helped, and that record is the history the loop reflects over [61]. SkillsVote attributes each outcome to skill, exploration, or environment before an update is admitted [63]. SkillOpt-Lite's consensus filter across rollouts is the form of credit assignment its ablation finds load-bearing [64]. In the general case, where a failed output may come from a stale memory entry, an under-specified rule, or a misrouted edge, the trace remains unsolved.

B. THE EDIT AND THE POOL

Who writes the candidate edit sets how often the local loop can start, and how the pool of artifacts is kept sets whether the gate can read what the edit did. Both decide whether the gate sees enough edits, and clean enough results, to be worth having.

Three kinds of author write the candidate edit. In the research systems an agent writes it by reflecting over the failure history, which is the author the loop of Section III assumes. In

the prompts-as-code platforms an engineer writes it at a desk, and session feedback never triggers a candidate (Section VII). A contributor writes it in the public skill repositories, where every substantive edit is human-authored and human-merged [65]. Who writes the candidate is not a gate, but where the author is a person, the local loop runs only as often as a person notices.

The pool the candidate enters is governed as carefully as the candidate. Eleven skill systems treat removing an artifact as an operation that must pass a check, the same as adding one [66]–[69] (Supplementary Material S6). The reason is in Ratchet's result: skills written by a language model gain nothing over a no-skill baseline until lifecycle hygiene is applied, while human-curated ones show a double-digit gain [70]. A pool nobody prunes starves the gate of signal, because failures caused by stale artifacts cannot be told from failures the edit under test would fix (Section X). Hygiene is not an alternative to descent; it is what makes the gate readable.

C. THE GATE: WHAT IT MEASURES, WHO DECIDES, HOW LONG IT WAITS

The local gate is the best-built step of the local loop, and a program now decides at it nearly everywhere a failure-driven loop runs. The gate is where an edit becomes evidence, so its form decides what a kept edit is worth.

On skills and orchestration, where the research activity is concentrated, the rule has four exceptions. On skills, SAGE and COSPLAY close both halves of the loop inside one research environment [33], [71]. The first exception is SkillChain, which puts a person after the judge: a human reflection gate before an edit reaches the skill bank [44]. SkillRL is the second: its success-rate threshold decides when a new skill is proposed, and nothing decides whether the skill stays [35]. The third is SkillForge, whose calibrated judge picks the replies to rewrite while nothing tests the rewrite [36].

On orchestration the systems went from search to a gate, and none closes both loops under lineage. The early ones searched for a topology under a score [72], [73], and the current ones keep a topology change only under an evaluation, one of them only when the change also satisfies a structural invariant [74]–[76]. The fourth exception, OptiMAS, has no gate at all: its optimizer reads the trajectories and the validation score and writes the next configuration, so the evaluation informs the edit without deciding it [77].

The counter-example is the most widespread artifact of all. Instruction files persist across IDEs and agents (Claude Code, Cursor, AGENTS.md [78], Windsurf, Continue.dev) and appear in a fifth to a quarter of coding-agent projects [79], [80]. Yet two controlled studies disagree on cost, and neither shows a gain in task success. In one, context files raise inference cost by about a fifth and buy nothing in task success [81]. In the other, a paired run on 124 pull requests toggles only the root AGENTS.md and finds that the file cuts median wall-clock time by about 29% and median output tokens by about 17% while leaving total tokens flat; the run does not measure correctness [82]. That is the patchwork stage of Section III on the most widely deployed substrate: a rule is added after a failure, nothing measures whether it helped, and the file grows.

Read by who decides, the three gate kinds of Section III fall on the local loop in a fixed pattern. A program decides nearly everywhere a failure-driven loop runs. A person decides only in curation, where an agent drafts and a person chooses what is kept, as in VerificAgent's fact-checked and frozen memories [83] and SkillChain's final reflection gate. Where no local loop exists, in the prompts-as-code platforms and the registries of Section VII, nothing decides at the local gate at all; what those systems gate is admission to a shared pool, which is a promotion decision. The pattern is close to the one Section IV predicts for the local half: a program wherever the loop has a check or an outcome to read, and a judge where the loop has only a judgment, calibrated against people in three systems and unmeasured elsewhere.

How long the gate waits is where the newer systems differ from the earlier ones. The earlier systems accepted an edit on one evaluation and absorbed the estimator's noise through the margin alone; the newer ones wait. SAGE-gate, a statistical acceptance gate unrelated to SAGE, commits an edit only when a paired per-item test says its wins are reliable [84]; in VaG, three heterogeneous critics must pass [85]; AutoSaddler feeds mini-batch failure signals into validation-based selection [58]. Waiting hides two different operations: if each of k independent proposals is wrongly accepted with probability p, requiring all k to agree drives the false-accept rate to p^k, while averaging k noisy scores shrinks the variance of the estimate only by $1/k$ (Appendix XIII). A gate that waits for a batch is the gate taking seriously the noise in its own measure of loss, and in the corpus that care arrived after the gate itself.

D. THE LEDGER: VERSIONED, NOT CORRECTED

In the last step of the local loop, the memory products have built the record of how each entry came to be, and not the correction that should act on it. The record matters twice: it is how the local loop reverses an edit that a later check shows made things worse, and it is what the promotion loop would read before letting an edit travel.

Letta's git-backed Context Repositories [86], Mem0's per-memory history [87], and Zep's bitemporal facts [88] all record how an entry came to be, so a user who asks what the system remembers and why can be answered. Research memory splits into write-gating and provenance on one side [89]–[92] and verifier-gated failure loops on the other [93], [94]. Only the governed collaborative memory of Cuadros et al. does both [95], which is why it alone reaches the top mark.

Five production backends still return facts that were revoked [96]: the ledger exists, the gate that would act on it does not. The gap sits in the observable-outcome and judged classes (C2, C3), where a revocation arrives as a user's objection rather than as a failed check. Rollback of a single edit inside one deployment exists, as edit-level reversal [67], [97] and as an admission test that rejects edits which cannot be undone [98]. The one negative result on it cuts deep: once a

contaminated skill has descendants, rolling it back recovers little, so gating must happen before commit [85]. No system rolls back a promotion that has crossed an organization, because none makes one (Section IX).

E. ALL FIVE CONDITIONS IN ONE SYSTEM

One system, SkillOpt, meets all five conditions of the local loop, and its stripped-down follow-up shows which of them carry the weight. That makes the conditions a description of something built rather than a wish list, and it separates the parts the loop needs from the parts that only steady it.

SkillOpt [31] treats one markdown skill document as the trainable external state of a frozen agent. A held-out exact-match split is the measurable loss, condition 1, an exact signal in the terms of Section IV. Rollout and reflection batches are the captured residuals of the failure log, condition 2. A consensus filter across rollouts is the credit assignment, condition 3. Add, delete, and replace edits under a bounded step are the candidate edit, condition 4. A held-out gate that accepts only a strictly better score is the gate, condition 5. Across six benchmarks, seven target models, and three harnesses it is best or tied on every cell its authors evaluate.

Its transfer across models, harnesses, and benchmarks measures how far one edit's lift carries, the evidence radius of Section III. That transfer is a measurement, not the gate that governs the edit's travel; in SkillOpt that gate is the held-out evaluator admitting the edit to the shared skill bank.

Its follow-up SkillOpt-Lite [64] strips two of its controls (a rejected-edit buffer and a slow update schedule), keeps the held-out gate and the consensus filter, and matches or beats it. So the gate and a consensus form of credit assignment are load-bearing, and the two it strips are stabilizers (Supplementary Material S5).

VII. THE PROMOTION LOOP IN THE INDUSTRY: HOW FAR AN EDIT TRAVELS, AND WHO LETS IT

The promotion loop exists in the industry only in parts, each built inside one operator, and no surveyed system carries a promotion out of its operator under lineage, access control, and rollback. That matters because the promotion loop is what lets a fix earned in one deployment reach another; without it, every customer relearns the same lesson. The section follows an edit upward, from a crossing inside one operator to the top rung, and ends on the grid that sets the local gate beside the promotion gate.

A. INSIDE ONE OPERATOR: WHAT CROSSES, AND THE CHECK WHERE IT ARRIVES

Every kind of crossing below the tenant boundary is occupied, each by a system of one operator, the organization that runs both sides with no customer boundary between them. Where such a system checks the arriving edit decides whether the receiving deployment ever learns that the edit helps there.

The crossings run up the ladder of Section III. Between users, ShareMem consolidates one user's memories for another and evaluates them there [99]. Between repositories, a rule in XRepoSkill survives only if it is verified in at least two of them [100]. Between teams, at the STACK rung, Evo2Team promotes routing skills and confirms them at the target [101]. The data-local clients of a federated run are partitions rather than deployments and so sit on no rung; across them, EvolveNet merges a shared harness only when the merge fixes at least as many held-out items as it breaks [102], and FederatedSkill's server agent applies, adapts, or rejects each client's edit and logs the decision [103].

A promotion differs from a push in having a check at all, and these systems differ in where the check sits. ShareMem, XRepoSkill, and Evo2Team check where the edit arrives; EvolveNet checks at the aggregator and nowhere else; FederatedSkill records a decision taken at the server. The receiving-side evaluation that Section III places at every rung is therefore present in the corpus and rare. Where it is present it is necessary rather than sufficient: nearly a fifth of Evo2Team's confirmed transfers still fail on graphs the confirmation never saw.

None of these leaves its operator: no system promotes across customers or organizations with evidence. That is the thesis of this article. Because the failure log and the trace of Section VI are hardly built, a system that tried would have few failure-driven edits to promote and no way to trace a failure on arrival back to the edit that caused it.

B. WHO LETS IT CLIMB: A PROGRAM, A PERSON, OR NOTHING

Who lets an edit climb is set by the feedback the crossing can get, so the corpus sorts into systems where a program admits the edit, systems where a person does, and one production system where nothing does. The sorting tells a team which promotion gate its own signal allows.

A program lets an edit climb where a check travels with it, as Section IV predicted. The research systems that promote into a shared bank or archive work this way: in NanoResearch, SkillOpt, DGM, ADAS, Voyager, and SAGE [29], [31], [33], [41], [104], [105] the same program that kept an edit admits it to the pool every later run draws from. In AlphaEvolve, as in FunSearch before it, the evaluator cascade that scores every candidate is the only thing that admits a program to the archive; AlphaEvolve is a research loop run inside Google rather than a product, and the programs kept in that archive went on to production gains [30], [106]. That pool is as far as any of them travels, and none has access control, audited promotion, or a gate at a receiving organization.

A person lets an edit climb where the signal is an observed outcome or a judgment. CASCADE consolidates memory under a human-and-agent gate [37]. Sierra routes agent-generated insights through human-authored expert answers [107]. In Devin one event serves both halves: the merge of a pull request is the outcome the local loop reads, and the reviewer's decision to merge is also what lets the edit stand in the repository every later session uses [32]. Spotify's automatically optimized prompts and plans go live only after an online A/B test and a launch decision [28]. In every one

of them the person at the promotion gate stands in for the exact signal production lacks, which makes the pairing of a program on the local gate with a person on the promotion gate the production-viable one rather than a compromise.

Nothing lets an edit climb in one production system, and nothing promotes at all in most research systems. Letta's Context Repositories push every edit to shared memory with lineage and no check; the ledger records what crossed, and nothing asks whether it should have. In the far larger group whose accepted edits never leave the deployment that made them, a program keeps the edit and nothing carries it anywhere; AutoAgent, whose analyzer revises one agent's own tool and peer descriptions, is among them [108].

C. THE TOP RUNG: CORE EXISTS, AND EVERYTHING IN IT IS ADMITTED BY HAND

Above the tenant boundary the scope exists and the promotion does not. CORE, the scope every tenant shares, exists today in two forms, a format and a registry, and an edit promoted into it on evidence is not one of them. CORE is the rung where cross-tenant promotion would happen, so what fills it now shows how much of the promotion loop the industry has built at the top.

The first form is a format. Claude Agent Skills, Cursor Rules, GitHub Copilot instructions, Windsurf, Continue.dev, Zed, and Replit all keep a git-tracked markdown artifact attached to a workspace, with an MCP-style tool layer beside it [80], [109], so an artifact written for one deployment can be carried to any other. Even where a repository runs an autonomous loop, the runtime state of that loop sits outside version control [110], so what the format versions is the artifact and not the process that edits it.

The second form is a registry that admits artifacts for everyone, and admission is by hand. In the five public skill repositories of one field study, every substantive edit is written and merged by a person, with one deprecation in 254 edits [65]. The MCP registries split into catalogues (namespaces and version records, no scanning) and control planes (scanning, access control, quarantine) [111]–[113]. The governance registries bring lineage and rollback [114]–[116]. One of them, EDDOps, gates promotion on evaluation thresholds before a human sign-off [114], but none generates the candidate from a failure.

The prompts-as-code platforms hold the rest of the machinery the promotion loop would need, and nothing feeds it. Braintrust, Vellum, and LangSmith Hub [117]–[119] keep immutable versions, evaluation thresholds, and rollback. But engineers author the candidates at a desk, an evaluation blocks a release on regression, and session feedback never triggers either, which makes them MLOps for scaffolds rather than for weights.

So tools and governance reach the top mark of Section V with no local loop at all.

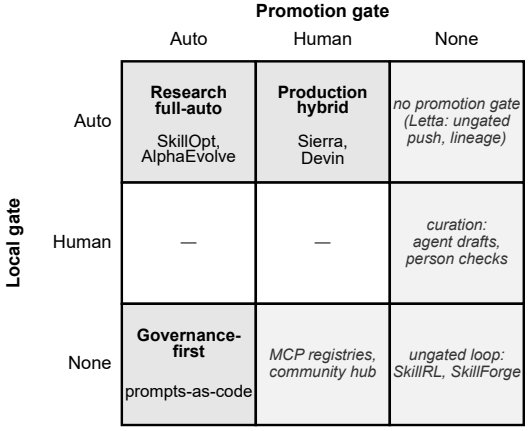


FIGURE 4. The two-loop design space. Rows give the local gate and columns the promotion gate, with None for a step that is absent or runs unchecked; a cell [local gate × promotion gate] such as [Auto × Human] pairs an algorithmic local gate with a human promotion gate. Darker cells mark the three populated kinds of system; lighter cells hold curation, the MCP registries and community hubs, and the systems whose promotion is ungated or absent.

D. THE TWO HALVES TOGETHER: THE GATE GRID

Set side by side, the local gate and the promotion gate show an industry that has converged on the first and is sorted on the second by the feedback it can get. The loop makes exactly two decisions, whether a local edit enters the scaffold and whether an accepted edit leaves the deployment, so a grid of the two gates places every system by both at once.

The grid of Fig. 4 puts the local gate in the rows and the promotion gate in the columns, each one of the three gate kinds of Section III. Every system whose local gate is a program sits in the top row, and what tells them apart is the column: a program, a person, or nothing. The column is not a free choice; it follows the signal as Section IV predicted, with one qualification the data adds for three systems.

Every [Auto × Auto] system whose promotion gate estimates the loss change runs on an exact signal or a simulated outcome, as EvolveNet's held-out merge and XRepoSkill's second repository do. Those on delayed or judged signals, NanoResearch, AutoManual, and ShareMem, decide by program on both halves, but by pruning, a judged rewrite, or a judged merge rather than by estimating a loss change. That is the degenerate gate Section IV describes for a loop without a per-edit estimate, and it is the qualification. [Auto × Human] holds observed outcomes and judgments and no exact-signal system. Nor does any system calibrate a judge under edit. Table 5 reads the grid cell by cell, with the signal found in each.

[Human × Auto] and [Human × Human] are empty: no surveyed system puts a person on the local gate of a failure-driven loop and then promotes the result. The human-gated row holds curation alone, where an agent drafts and a person decides what is kept.

The bottom row holds the systems with no failure-driven

TABLE 5. The gate grid read cell by cell: who keeps a local edit and who lets it climb, which systems sit in the cell, and the feedback signal they gate on. [Human × Auto] and [Human × Human] hold no surveyed system.

Cell	Who keeps the local edit, who lets it climb	Who is there	Signal found there
[Auto × Auto]	A program keeps it; a program lets it climb	Research systems that close both halves inside one environment or one operator	Exact and simulated outcomes; delayed and judged only under pruning or merging
[Auto × Human]	A program keeps it; a person lets it climb	Production systems with a reviewer on promotion, and CASCADE	Observed outcomes and judgments
[Auto × None]	A program keeps it; nothing lets it climb	Letta's ungated push; systems that promote nothing	As on the local gate
[Human × None]	A person keeps it; nothing lets it climb	Curation	Judged
[None × Auto]	No failure triggers a candidate; a threshold gates release	Prompts-as-code platforms and governance registries	No failure-driven loop
[None × Human]	No failure triggers a candidate; a person admits	Open standards, MCP registries, community hubs	No failure-driven loop
[None × None]	Nothing checks or promotes the edit	SkillRL and SkillForge, unchecked skill loops	Outcomes and judgments

loop, which enter the grid for comparison rather than through the audit (Supplementary Material S4). They are the prompts-as-code platforms, which gate release on evaluation but generate no candidates, and the registries and community hubs, Buildrix among them [120], in which a person registers an artifact and a person, or an admission scan, approves its use. SkillRL and SkillForge, the two failure-driven skill loops that run unchecked, sit in the row's last cell.

Of the six systems that promote without lineage on the promoted artifact (Fig. 3), the tier rule of Section V admits the evidence for five, Spotify among them, and these join cells already occupied and open no new one (Supplementary Material S2). No system has both the learning of the top row and the governance of the bottom, and what no cell contains is governed cross-tenant promotion, which Section IX specifies.

VIII. FIVE RESEARCH LINES ARRIVE AT THE SAME LOOP

Nobody designed the loop this survey describes: five lines of research built it one part at a time, each while solving a different problem, and that is why it exists in pieces that nobody has assembled. Where the parts came from explains the shape the corpus shows, in which the steps some line of research worked on are built and the steps no line started from are missing. It also checks the survey's starting premise from outside, because lines working independently of one another begin by treating the scaffold and the weights as separate things to adapt.

Prompt optimization built the candidate edit, condition 4 in the list of Section III. A theory of learning in context built the penalty inside the gate, condition 5. Harness engineering built the trace from a failure to its cause, condition 3. Skill-library maintenance took the gate and turned it around, so that the gate decides when a skill is retired as well as when one is admitted. Transfer and federation built the step after the gate, the promotion loop: how far an edit travels and what checks it when it arrives. Table 6 sets the five side by side with the problem each started from and what each leaves open.

Reflective evolution came from prompt optimization and continual learning. It proposes edits by reflecting over trajectories, with a two-timescale argument for keeping them outside the weights [4], [121]. Reflective context learning (RCL) came from asking why learning in context overfits and forgets as learning in weights does, and its generalization penalty is the complexity term Ω of the gate [122]. Harness optimization grew out of orchestration, and it names the trace as its bottleneck [59]–[62], [75], [123], [124]. The lifecycle line started from skill pools that bloat under platform limits and turned the gate around to face removal; once those limits bind, the complexity penalty becomes a hard frontier [63], [66]–[70], [125]. The transfer line came from asking how far an edit travels. Its work runs from measuring one edit's reach a task at a time, through confirming a transferred skill at the receiving team, to federating edits across clients [34], [42], [101]–[103], [126], [127].

None of the five started from the first two of the five conditions, the measurable loss and the failure log. Each line took its loss ready-made from the benchmark it inherited, and logging failures was nobody's starting problem. That is why Section VI finds the loop built from the gate backward. The parts with a research line behind them are built; the trace is built, but only one substrate at a time; and the parts with no line behind them are not.

Two things follow from the convergence. The first is that the split between scaffold and weights is confirmed independently: GEPA, Meta-Harness, MOSS, RCL, PACE, and the harness-optimization papers all begin from the observation that weights and the external artifact layer are distinct adaptation surfaces with different cost, reversibility, and scope. A system that updates both keeps them as separate levers [128]. The second is a gap. No line documents a governed

TABLE 6. Five research lines converging on artifact-layer descent: the problem each started from, the part of the loop it built, and what it leaves open.

Research line	Started from	The part of the loop it built	What it leaves open
Reflective evolution	Prompt optimization and continual learning	Condition 4, the candidate: proposal by reflection over trajectory evidence; the two-timescale case for editing outside the weights	A persistent artifact; a bounded step; a held-out gate
Reflective context	Learning in context as a setting of its own	Condition 5, the penalty: the learned object in context space; a generalization penalty, the Ω of the gate	A cross-deployment promotion loop
Harness optimization	Orchestration and harness engineering	Condition 3, the trace: attribution of a failure to one slot of the orchestration graph; source-level edits	Attribution across substrates; a governed promotion loop; access control; rollback
Lifecycle and multi-objective	Skill pools that bloat under platform limits	The gate on removal: gated retirement; hygiene as a precondition; a hard frontier as the shape of Ω	Closing the loop as descent; retiring a promoted artifact's descendants
Transfer and federation	How far an edit travels	The promotion loop, the check on arrival: evidence radius measured one task at a time; cross-client aggregation; receiving-side confirmation	A cross-organization gate, lineage, and rollback

promotion loop; Section IX scores the two that come closest, FederatedSkill and EvolveNet.

What does not follow is that the shape is the best one. The vendors' convergence on one artifact format and one tool layer is the same kind of evidence from the production side (Section VII), reached by groups that were not looking for it. Convergence shows only that the same constraints keep producing the same shape.

IX. THE MISSING ARCHITECTURE: GOVERNED CROSS-TENANT SCAFFOLD OPTIMIZATION

The missing piece of the loop is governed cross-tenant promotion, the step that carries an edit from one organization to another under a gate, and four properties make it specific enough to audit for. Named as four properties, the gap becomes something a reader can check any system against, their own included, and no surveyed system holds all four.

The four properties are these:

(a) **auto-gated local loop**: the commit criterion is an algorithmic threshold;
(b) **cross-organization promotion**: the promotion loop routes artifacts between organizations (we use *cross-tenant* and *cross-organizational* interchangeably);
(c) **versioned lineage with access control**: promoted artifacts carry versions, audit trails, and role-based access policies;
(d) **rollback**: a bad promotion can be reverted without redeploy.

All four describe the artifact: how it is made and how far it may travel. Receiving-side evaluation, the check without which no promotion is safe, is not among them, because it is a property of the gate where the edit arrives rather than of the artifact. Section VII records where that check exists, and the architecture below includes it as its third part.

The closest approaches split the four properties among them. No row of Table 7 is full on promotion across organizations.

The partition is sharp. The two federated systems carry the automatic gate and part of the promotion. SkillForge carries a calibrated judge and part of the lineage, but no acceptance test and no promotion. The registries carry lineage and rollback without a local loop, and the control planes carry access control alone. Rollback exists inside one federated round and, as Section VI records, inside single deployments; no system rolls back a promotion that has crossed an organization boundary, because none makes such a promotion (Supplementary Material S4).

In machine-learning terms, what is missing is a cross-tenant aggregator with an automated evaluation gate at the scaffold layer. It would carry five parts: per-tenant artifact pools, each tagged with provenance; an aggregation step across tenants that respects tenant isolation; a held-out evaluation at the receiving tenant that blocks a promotion whenever it does not lower that tenant's patch loss; versioned lineage with access control; and a rollback envelope on every promoted artifact. The federated-learning analogy [129]–[131] is useful only in the limit where artifacts are mergeable update representations. Federated prompt tuning, in which participants learn a shared prompt rather than shared weights [132], is the weight-side precedent closest to that limit. In surveyed practice the promotion loop is selection and merge: the nearest systems aggregate program edits with a language model, not by averaging.

Whether the architecture *should* be built depends on the feedback-signal class, not on the application domain (Section IV). Where the signal is an exact check, automatically gated cross-tenant promotion of a test-validated skill would be low-risk, and receiving-side evaluation is cheap to add. Where it is an observed outcome, the sending side's outcome does not transfer, so promotion should be automated only

TABLE 7. The closest approaches to governed cross-organization scaffold optimization, scored against the four properties.

System	(a) auto-gated local loop	(b) cross-organization promotion	(c) versioned lineage with access control	(d) rollback
FederatedSkill [103]	full	partial: privacy-preserving aggregation of skill edits across heterogeneous clients that sit under one operator	partial: auditable per-modification decision log	none
EvolveNet [102]	full	partial: held-out merge gate at the aggregator, but clients are benchmark partitions, not organizations	none: no lineage, no access control	full: rollback to the previous shared harness after a failed round
SkillForge [36]	none: a calibrated judge selects the failures, nothing tests the rewrite	none: skill versions stay in the scenario deployment	partial	not documented
Governance-first registries [114]–[116]	none: no inner loop	none	partial: lineage without access control	full, with evidence
MCP control planes [111]	none: no inner loop	none	partial: access control without lineage	not reported

behind receiving-side evaluation. That check re-establishes the lift under the receiver's own measure, and it would catch a scaffold over-fitted to one deployment before it propagates to another. Even that check is necessary rather than sufficient: some of Evo2Team's transfers still fail on new graphs after target-side confirmation [101].

Where the signal is a judgment, a human promotion gate is a design feature, and governance means lineage and rollback around a human decision. With a delayed reward, property (a), the auto-gated local loop, cannot be met; what remains is lineage and rollback without an automatic gate. Safety-critical domains, the usual case for human gates, are judged or delayed domains with high stakes. Whatever the class, the measure bound of Section III holds: where tenants measure different things, as they usually do across organizations, no lift transfers without the receiving-side check.

X. THE FAILURE SURFACE

A deployment that runs without the loop, or with only half of it, pays in seven failures of the artifact layer, and each failure is answered by a piece of the loop that patchwork leaves out. That matters because it makes the failures a cost of patchwork rather than a new threat model, and it puts the remedy in the loop itself rather than in a further layer of defense.

Table 8 pairs each failure with the piece of the loop that would close it. The entries are documented attacks and degradations, the scaffold-layer entries in the technical-debt ledger that Sculley et al. opened for machine-learning systems a decade ago [133]. Six are shown below in a measured case; the seventh, patch overfitting, follows from the measure bound rather than from a measurement. The table's last column records the feedback-signal class in which each failure dominates. Wherever the third prediction of Section IV names a class's failure, the table agrees: bloat in the exact class, poisoning in the observable-outcome class, evaluation fragility (the gamed judge) in the judged class, and plasticity loss (the drift) in the delayed class.

Bloat is not only slow; it is a signal-quality problem. Instruction-following accuracy falls and latency grows sharply as the instruction count scales [134]. Ratchet's result is the loop-level symptom: model-written skills gain nothing until lifecycle hygiene is applied [70]. In the vocabulary of Section III, failures cannot be captured and clustered on a messy artifact pool, which is why gated retirement comes before descent, not after it.

Poisoning and injection are the two ways an adversary reaches the scaffold, and both arrive through the channel the observable-outcome class depends on. Memory is reached by writing: attacks against standard retrieval-augmented generation succeed at high rates [135], AgentPoison shows that backdoors can be planted in long-term memory or knowledge bases [136], and memory control-flow attacks work across the major agent frameworks [137]. Tools are reached by output: a tool's result or a connected server can smuggle instructions into context [138], [139], and one approved MCP configuration could later be modified without re-approval (CVE-2025-54136) [140]. The responses come from the ledger of Section VI and the governance of Section IX: provenance and write gating with a rollback envelope on memory [141], [142], and a privilege hierarchy over instructions [143] with tool bindings that pass the same promotion gate as every other coordinate.

Evaluation fragility compounds because the evaluation suite is itself a patch-local artifact rather than a fixed instrument. It is optimized against: once a metric becomes the target it is gamed, optimizing a proxy reward past a point lowers the true score [38], and access asymmetries alone produce large relative gains that reflect exposure to the test [144]. It is read non-independently, because the gate scores path-dependent agent trajectories whose readings do not compose

TABLE 8. The failure surface of patchwork scaffold maintenance: what goes wrong, the piece of the loop that answers it, and the feedback-signal class in which it dominates.

Failure	What goes wrong	The loop piece that answers it	Dominant class
Scaffold bloat	Rules and skills accumulate until the model follows them worse and the gate has no clean signal	Complexity penalty Ω in the gate; pruning and expiry as first-class gated operations	C1, C3
Poisoned memory	An attacker writes an entry that later retrieval trusts	Provenance on every entry; write gating; a rollback envelope	C2
Injection via tools and MCP	A tool's output or a connected server carries instructions into context	Narrow tool permissions; a privilege hierarchy over instructions; tool bindings pass the same promotion gate as any other coordinate	C2
Evaluation fragility	The suite that gates edits is itself edited, gamed, and read non-independently	Held-out evaluations scoped within evidence radius, weighted toward recurring modes, re-read as the target drifts, checked against attribution	C3 (primary), C1
Coordination failures	Multi-agent roles blur and context is lost between agents	Orchestration topology treated as a scaffold coordinate under the same gate and rollback discipline	C2
Patch overfitting	A scaffold tuned to one deployment degrades another	Evidence-radius discipline at promotion; receiving-side held-out evaluation	C2
Plasticity loss	Old rules, polluted memory, and drifting tool versions crowd out adaptation	Expiry and rollback as explicit loop operations rather than retrospective hygiene	C4

like ordinary test-set statistics. The suite also goes stale, because it adapts on the same cadence as the scaffold it gates. Because failures concentrate in a finite catalogue of recurring modes, a suite that samples tasks at random underweights exactly the families that drive the failure rate. The consequence has been measured: harness optimizers cannot be told from trial-and-error by downstream score alone [43]. The response follows from those facts: a suite that is held out and scoped, weighted toward recurring modes, re-read as the target drifts, and checked against attribution rather than only against score.

The remaining three are failures of structure rather than of signal. Multi-agent systems fail in fourteen measured modes, dominated by role confusion and context lost between agents [145], and the response is to treat the topology as one more scaffold coordinate under the same gate and rollback. A scaffold tuned to one deployment over-fits it by construction (Supplementary Material S3). Because the measure bounds the lift, the response at promotion is evidence-radius discipline, with a held-out evaluation at the receiving side. And a scaffold that only grows loses plasticity the way a long context does: context rot [146], instruction conflict, memory pollution, and tool-version drift accumulate until the only way to recover it is expiry and rollback, run as loop operations rather than as retrospective hygiene.

The surface stays largely unmitigated in production because no surveyed system runs the pieces in the table together. What the table itemizes, then, is the cost of running without the loop, already measured.

XI. OPEN PROBLEMS AND CONCLUSION

Each feedback class leaves one problem between the industry and the assembled loop, and governed promotion across organizations leaves a fifth. That matters because it makes the remaining work specific rather than a general agenda: each problem is the step its class is blocked on.

1. **Exact signals: credit assignment from a failing check to a scaffold coordinate.** A failing check has to be traced to the scaffold coordinate that caused it, and the loop has no general way to do that. A test tells the loop that an output failed, not which instruction, skill, or tool binding caused it. In the observable-outcome class the gap is wider still, since the outcome says only that the episode failed. Slot attribution [62], intervention-based attribution [74], and contrastive multi-agent attribution [60] each solve one substrate's case; a general method is the bottleneck the harness-optimization family names.
2. **Observable outcomes: receiving-side evaluation before promotion.** A promoted edit has to earn its lift again where it arrives. The outcome that justified it on the sending side does not transfer, and even where a promoted edit is checked on arrival, some transfers still fail (Section VII). The gate a federated scaffold optimizer needs, a held-out test at the receiving tenant, has been proposed but not built [147].
3. **Judged signals: calibrating a gate that is itself being edited.** The gate in a judged deployment has to stay calibrated against people while the loop edits the rubric it judges by. When the rubric is a scaffold coordinate, each accepted edit to it moves the instrument that judges the next edit. No method that co-evolves a rubric or judge reports agreement with human judgment, apart from RLPR,

whose reward is a probability rather than a rubric. No scaffold system calibrates a judge under edit (Section IV), and the nearest measurement is on the human side, where evaluators' criteria drift as they grade [39].

4. **Delayed signals: telling descent from search.** When the reward arrives too late to judge each edit, the loop needs another way to show that its gains come from descent rather than search. Where the outcome arrives after many edits, an optimizer's gains cannot be distinguished from trial-and-error by outcome alone [43]; the open method is to evaluate harness optimizers against their attribution claims rather than their scores.
5. **The governed cross-organization loop.** Promotion across organizations needs an architecture that no surveyed system documents. Section IX states that architecture as properties (a) through (d). Without the receiving-side gate of problem 2, cross-organization promotion cannot be safe in the two classes where it should be automatic, the exact and the observable-outcome classes.

The field has built an external adaptation layer for language models but has not, as far as public evidence shows, assembled the optimizer that should govern it. Both earlier papers of this series supply the hypothesis for why the layer belongs where it is: errors concentrate in a few key decisions; inside a bounded deployment, those decisions fail in a small and recurring catalogue of modes; and such a catalogue can be fixed one mode at a time in a persistent, inspectable, reversible artifact outside the weights. The corpus is consistent with that hypothesis. If it holds, the gated and scoped artifact layer is not a workaround but the architecture the error structure calls for.

Seen whole, the corpus says the industry has built most of that optimizer piecemeal. The local loop closes automatically in most research systems and in a growing number of production ones; evidence is batched before acceptance; retirement is gated; memory ships with its lineage; formats and registries have converged; and the first federations promote edits across clients inside one operator. What nobody has done is assemble the pieces. No surveyed system promotes a validated edit across an organization boundary under an automatic gate with lineage, access control, and rollback. Read by signal class, as Section IX does, this is not one gap but four, each with its own reachable answer. Under every class the same rule holds: a lift travels only as far as its measure does.

The frontier model generalizes. The local scaffold specializes. Reliability comes from governing that specialization. Because the gates that judge each edit are themselves part of the moving scaffold, the same evidence-radius and lifecycle discipline the optimizer needs must extend to the gates.

XII. APPENDIX: FORMAL STATEMENT OF ARTIFACT-LAYER DESCENT

This appendix states the setup behind Section III, a one-step descent inequality under a noisy gate, an informal convergence proposition, the optimization-analogy correspondence, and four worked cases. No new theorem is claimed; the proposition is the ordinary target of local stochastic optimization under noisy estimators, restated for a discrete, versioned parameter.

A. SETUP

Let the frontier model have fixed parameters θ_M and let the scaffold be a structured object $\theta_s \in \Theta_s = \Theta_1 \times \cdots \times \Theta_6$ over the six substrate coordinates of Section V; Θ_s is discrete, symbolic, and versioned. For a deployment D, $L_D(\theta_s) = \mathbb{E}_{x\sim D}[\ell(\pi(\theta_M, \theta_s, x))]$. A candidate edit $z_t \sim Q_t(z \mid H_t, \theta_s^t)$ is drawn from a proposal process conditioned on the accumulated history H_t of failures, corrections, rejected outputs, evaluation regressions, and prior accepted edits. Q_t is not uniform: for a mode m that recurs with frequency f_m and carries per-occurrence loss s_m, its expected contribution to the failure rate is $f_m s_m$, so high-frequency or high-severity modes are proposed first. The finite directional difference is $\Delta_z L_D(\theta_s) = L_D(\theta_s \oplus z) - L_D(\theta_s)$; the gate sees only a noisy estimate $\widehat{\Delta}_z L_D$. The update is

$$\theta_s^{t+1} = \theta_s^t \oplus z_t \text{ if } G_t(z_t) = 1, \quad \theta_s^{t+1} = \theta_s^t \text{ otherwise.}$$

B. DESCENT INEQUALITY

Assume (A1) positive descent bias on accepted edits, $\mathbb{E}[\widehat{\Delta}_{z_t} L_D \mid G_t = 1, \theta_s^t] \le -\gamma_t - \tau_t$ for some $\gamma_t > 0$, and (A2) bounded estimator bias on accepted edits, $|\mathbb{E}[\Delta_{z_t} L_D - \widehat{\Delta}_{z_t} L_D \mid G_t = 1, \theta_s^t]| \le \varepsilon_t$. With $p_t = \Pr(G_t = 1 \mid \theta_s^t)$,

$$\mathbb{E}[L_D(\theta_s^{t+1}) \mid \theta_s^t] \le L_D(\theta_s^t) - p_t(\gamma_t + \tau_t) + p_t\,\varepsilon_t.$$

Per-step expected improvement is the gate's margin minus its estimator bias, weighted by acceptance probability. In the feedback-signal classes of Section IV, ε_t is near zero for the check itself in C1, with the sampling error of a finite suite over the patch remaining, bounded by batching for an observable outcome (C2), equal to the judge's bias and bounded only by calibration for a judged signal (C3), and unavailable per edit for a delayed signal (C4).

C. CONVERGENCE (INFORMAL)

Add (A3) proposal coverage with acceptance: at every non-gate-stable scaffold, Q_t places probability at least $q > 0$ on an admissible edit with $\mathbb{E}[\Delta_z L_D] \le -\delta$, and the gate accepts such an edit with probability at least $q' > 0$, so that $p_t(\gamma_t + \tau_t) \ge qq'\delta$ there; (A4) effective compactness of the reachable orbit; and (A5) summable bias, $\sum_t p_t \varepsilon_t < \infty$; and let $0 \le L_D \le B$. The one-step inequality is then of the Robbins–Siegmund form, a non-negative process that decreases in expectation up to a summable perturbation $p_t\varepsilon_t$, so $L_D(\theta_s^t)$ converges almost surely and $\sum_t p_t(\gamma_t + \tau_t) < \infty$ almost surely [148]. The finite sum carries the content: a bounded loss cannot pay for infinitely many accepted edits with a positive expected margin, so under (A3) the process visits non-gate-stable scaffolds only finitely often, and with (A4) the limit set of $\{\theta_s^t\}$ is gate-stable: no admissible edit clears the gate with negative expected directional loss in the limit. This is convergence of the loss and stability of the

TABLE 9. Optimization concepts and their scaffold analogues.

Optimization concept	Scaffold analogue
Parameter vector	Instructions, skills, memories, tools, routing graphs, and governance rules, evaluation gates among them
Loss	Failure, user correction, evaluation regression, retrieval miss, tool error, schema violation
Gradient estimate	Credit assignment identifying the coordinate responsible for a failure
Step	Accepted scaffold edit; step size is edit magnitude under $\oplus$
Aggregated update	Several recurring failures combined before an update, by averaging or by agreement across k proposals
Update radius	Promotion scope: which loss is descended (USER, PROJECT, STACK, TENANT, CORE)
Validation loss	Held-out patch evaluation before merge
Regularization	Complexity penalty $\Omega(z)$, bloat control, cross-patch regression checks
Overfitting	Scaffold improves on this deployment and degrades elsewhere
Rollback	Reverting a harmful accepted update

gate decision, not convergence to an optimum; a globally suboptimal gate-stable scaffold is consistent with the result, because Q_t may never propose the improving edit or $\widehat{\Delta}$ may be biased to reject it. In a non-stationary deployment ($D = D_t$) the same mechanism tracks a moving optimum rather than converging once, which is why recency weighting, pruning, expiry, and rollback are the scaffold-level analogues of continual-learning machinery rather than optional hygiene.

D. CORRESPONDENCE

Table 9 pairs each optimization concept with its scaffold analogue. Credit assignment here is attribution, not back-propagation. Scaffold systems do not differentiate through instruction files. They trace an output error to an upstream coordinate, manually or with a judge or verifier. The descent argument needs only that this attribution succeeds with non-zero probability per failure, which is the credit-assignment condition of Section III.

E. WORKED CASES

Retrieval. Failures recur because retrieved context omits the decisive clause. Attribution traces the error to the retrieval coordinate: chunking, query rewriting, metadata filter, embedding model, reranker, or context-packing rule. The candidate edit tightens chunk boundaries or adds a metadata constraint, and the gate checks that recall and answer accuracy improve on the patch evaluation without degrading adjacent cases. Signal class C1 or C2.

Pitch review. The system repeatedly overrates decks that claim a large market without numbers, and the failure cluster points at an under-specified rubric coordinate. The edit changes ''evaluate market size'' into ''require a numeric estimate and a source; if absent, cap the score''; the gate runs the revised rubric against held-out decks with human scores. What the edit changed was the gate that will judge the next edit: the reflexive gate of Section IV, class C3.

Tool use. Calls fail because optional parameters are under-specified. The responsible coordinate may be the tool schema, the call policy, or the clarification rule. The edit adds required fields or a pre-call uncertainty check; the gate checks that tool-call failures fall without raising refusals or latency. Class C2.

Memory. A stale customer preference is repeatedly retrieved and causes bad decisions. The responsible coordinate is a memory entry together with its provenance, priority, and expiry policy. The edit corrects, expires, or version-tags the entry; the gate checks that future behavior improves without erasing useful history. Class C2, with the correction itself observable only when a later outcome reveals it.

XIII. APPENDIX: GATE FAMILIES AND FEEDBACK-SIGNAL CLASSES

Section III collapses every production gate into one per-edit test G_t on an estimate of the loss change. Surveyed systems instantiate that test in several distinct ways, and some of them are better read as proposal pruning that precedes the per-edit gate than as estimators of $\widehat{\Delta}L_D$. Table 10 lists the families, the slot each occupies, representative systems, and the feedback-signal class (Section IV) each family serves.

The first five families populate G_t directly and, under the assumptions of Appendix XII, inherit its descent guarantee. Semantic merge and archive selection are not estimators of $\widehat{\Delta}L_D$; they constrain the proposal distribution, and the descent argument applies to the composition of pruning and gate. Multi-sample agreement is listed because it is the correct analogue of ''wait for k pieces of evidence before accepting an edit'', a different operation from mini-batch averaging and one that should not be conflated with it. Read down the signal-class column. The only families available to a judged (C3) deployment are the two whose estimator is a judge. A delayed (C4) deployment has no family that estimates the loss change per edit at all: what it has is semantic merge before the gate and whatever proxy milestone stands at the horizon. That is the structural content of Section IV.

ACKNOWLEDGMENT

The authors used an AI assistant (Claude, Anthropic) to help search the literature, check citations, and edit prose; all claims, scores, and classifications were reviewed and approved by the authors. Earlier versions of this argument

TABLE 10. Gate families, their slot in the formalism, and the feedback-signal class each serves.

Gate family	What it estimates	Slot in Section III	Signal class	Representative systems
Surrogate verifier	Pass or fail from tests, a symbolic checker, or an exact match	Per-edit gate G_t	C1	DGM (sandboxed compile-and-edit check; score weights selection, not retention), SkillOpt (held-out exact match), Voyager (game-state check: C2)
Evaluation cascade	$\widehat{\Delta}L_D$ on a held-out task suite, cheap-to-expensive evaluators	Per-edit gate G_t	C1 when the suite is exact; C2 when it is a task suite	AlphaEvolve, Braintrust, Vellum, LangSmith Hub
RL threshold	Reward improvement against a deployment threshold	Per-edit gate G_t	C2	SAGE, PACE
LLM judge with calibration	Pairwise or rubric score, calibrated against human agreement	Per-edit gate G_t	C3	SkillChain (judge stage, before its human reflection gate)
Reviewer judgment	Human credit assignment by pull-request review, expert sign-off, or a user's acceptance	Per-edit gate G_t, human-instantiated	C3 (and C2 when the reviewer has outcome data)	Sierra Agent OS 2.0, Anthropic skills repository, CASCADE (a person and an agent approve consolidation into shared memory)
Semantic merge	Whether a candidate duplicates or conflicts with existing artifacts	Proposal pruning before G_t	C4 (also a pre-filter in C2, C3)	NanoResearch orchestrator, FederatedSkill server agent, EvolveR (admission: cosine pre-filter plus LLM equivalence judgment on each distilled principle; retention: periodic pruning on per-principle success rate)
Archive selection	Whether a candidate improves on, or diversifies, an archive of already-scored programs	Proposal pruning before G_t	Any; inherits the class of the evaluator that scored the archive (C1 in AlphaEvolve)	AlphaEvolve (MAP-Elites-style database), ADAS
Multi-sample agreement	Whether k independent proposals or evaluations agree; false-accept rate decays as p^k rather than variance as $1/k$	Aggregation filter on the estimate	C2, C3 noise absorption	VaG (three heterogeneous critics must all pass); SkillOpt-Lite (consensus filter across rollouts)

appeared as an arXiv preprint and as a non-archival workshop submission; this article extends them with the feedback-signal classification of Section IV and a corpus scored through October 2026.

REFERENCES

[1] M. Zaharia, O. Khattab, L. Chen, J. Q. Davis, H. Miller, C. Potts, J. Zou, M. Carbin, J. Frankle, N. Rao, and A. Ghodsi, "The shift from models to compound AI systems," Berkeley Artificial Intelligence Research (BAIR) Blog, https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/, 2024.

[2] R. Kirk, I. Mediratta, C. Nalmpantis *et al.*, "Understanding the effects of RLHF on LLM generalisation and diversity," in *ICLR 2024*, 2024.

[3] V. Padmakumar and H. He, "Does writing with language models reduce content diversity?" *arXiv preprint*, 2023.

[4] R. Tiwari, K. Sareen, L. A. Agrawal, J. E. Gonzalez, M. Zaharia, K. Keutzer, I. S. Dhillon, R. Agarwal, and D. Khatri, "Learning, fast and slow: Towards LLMs that adapt continually," *arXiv preprint*, 2026.

[5] S. Dohare *et al.*, "Loss of plasticity in deep continual learning," *Nature*, vol. 632, no. 8026, pp. 768–774, 2024.

[6] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *arXiv preprint arXiv:2106.09685*, 2021.

[7] M. L. Arbuzov, S. Bei, Z. Dong, D. Kalaev, and A. A. Shvets, "Beyond exponential decay: Rethinking error accumulation in large language models," *arXiv preprint*, 2025.

[8] M. L. Arbuzov, L. Mosbacker, S. Bei, Z. Dong, D. Kalaev, and A. Shvets, "The architecture of errors: From universal impossibility to patch-local LLM reliability," *arXiv preprint arXiv:2605.30628*, 2026.

[9] T. Ding, A. Nannapaneni, B. Liu, and L. Zhang, "Always-on agents: A survey of persistent memory, state, and governance in LLM agents," *arXiv preprint arXiv:2606.30306*, 2026.

[10] Z. Lin, X. Hao, R. Fu, S. Cui, K. Chen, C. Li, Z. Li, and F. Xiong, "A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle," in *Proceedings of EMNLP 2026*, 2026, arXiv:2604.16548v3, 22 Sep 2026.

[11] H.-a. Gao, J. Geng, W. Hua, M. Hu, X. Juan *et al.*, "A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence," *Transactions on Machine Learning Research*, 2026, arXiv:2507.21046v4, 16 Jan 2026.

[12] J. Fang, Y. Peng, X. Zhang, Y. Wang, X. Yi, G. Zhang, Y. Xu, B. Wu, S. Liu, Z. Li, Z. Ren, N. Aletras, X. Wang, H. Zhou, and Z. Meng, "A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems," *arXiv preprint arXiv:2508.07407*, Aug. 2025.

[13] H. Zhou, H. Hu, T. Luo, Y. Shang, C. Fang, Z. Chen, L. Xiao, and Q. Zhang, "Self-evolving coding agents," *arXiv preprint arXiv:2608.03392*, Aug. 2026.

[14] J. Guo, Z. Hao, C. Wang, C. Fan, T. Luo, H. Li, Y. Gao, H. Mei, J. Peng, R. Xu, M. Dong, H. Wu, M. Zheng, K. Han, S. Wang, C. Xu, and Y. Wang, "From question answering to task completion: A survey on agent system and harness design," *arXiv preprint arXiv:2606.20683*, Jun. 2026.

[15] B. Xia, L. Zhu, Z. Xing, Q. Lu, D. Sejdinovic, and X. Xu, "Harnessing agent skills: Architectural patterns and a reference architecture for skill-mediated LLM agents," *arXiv preprint arXiv:2606.20631*, May 2026.

[16] K. Ding, Y. Zhou, C. Jin, F. Tong, M. Zhou, and D. N. Metaxas, "Agent skill evaluation and evolution: Frameworks and benchmarks," *arXiv preprint arXiv:2606.11435*, Jun. 2026.

[17] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li, D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, and S. Liu, "A survey of context engineering for large language models," *arXiv preprint arXiv:2507.13334*, Jul. 2025.

[18] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2309.03409.

[19] Q. Guo, R. Wang, J. Guo, B. Li, K. Song, X. Tan, G. Liu, J. Bian, and Y. Yang, "EvoPrompt: Connecting LLMs with evolutionary algorithms yields powerful prompt optimizers," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2309.08532.

[20] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel, "Promptbreeder: Self-referential self-improvement via prompt evolution," *arXiv preprint arXiv:2309.16797*, 2023.

[21] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling declarative language model calls into self-improving pipelines," *arXiv preprint arXiv:2310.03714*, 2023.

[22] M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, Z. Huang, C. Guestrin, and J. Zou, "TextGrad: Automatic "differentiation" via text," *arXiv preprint*, 2024.

[23] Q. Zhang, C. Hu, S. Upasani *et al.*, "Agentic context engineering: Evolving contexts for self-improving language models," arXiv preprint arXiv:2510.04618, 2025.

[24] W. Zhou, Y. Ou, S. Ding, L. Li, J. Wu, T. Wang, J. Chen, S. Wang, X. Xu, N. Zhang, H. Chen, and Y. E. Jiang, "Symbolic learning enables self-evolving agents," *arXiv preprint arXiv:2406.18532*, 2024.

[25] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai, "Self-taught optimizer (STOP): Recursively self-improving code generation," in *First Conference on Language Modeling (COLM 2024)*, 2024, arXiv:2310.02304.

[26] M. Robeyns, M. Szummer, and L. Aitchison, "A self-improving coding agent," *arXiv preprint arXiv:2504.15228*, 2025.

[27] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS 2023), Datasets and Benchmarks Track*, 2023, arXiv:2306.05685.

[28] E. Palumbo, A. Tamborrino, V. Ode, B. Lacker, A. Casas Escoda, J. Hopple, M. Better, J. Leoni, H. Galvão, H. Bouchard, M. Lalmas, J. L. Redondo García, A. Abebe, A. Clifton, A. Blomberg, H. Lindström, D. Doro, and C. Doig Cardet, "Bootstrapping conversational recommendation agents at spotify: Synthetic data generation and self-improvement loops," *arXiv preprint arXiv:2609.30297*, 2026.

[29] J. Zhang *et al.*, "Darwin Gödel machine: Open-ended evolution of self-improving agents," *arXiv preprint*, 2025.

[30] A. Novikov *et al.*, "AlphaEvolve: A coding agent for scientific and algorithmic discovery," arXiv preprint arXiv:2506.13131, 2025.

[31] Y. Yang, Z. Gong, W. Huang, Q. Yang, Z. Zhou, Z. Huang, Y. Li, X. Gao, Q. Dai, B. Liu, K. Qiu, Y. Yang, D. Chen, X. Yang, and C. Luo, "SkillOpt: Executive strategy for self-evolving agent skills," arXiv preprint arXiv:2605.23904, 2026. [Online]. Available: https://microsoft.github.io/SkillOpt

[32] Cognition, "Devin's 2025 performance review: Learnings from 18 months of agents at work," https://cognition.com/blog/devin-annual-performance-review-2025, 2025.

[33] J. Wang *et al.*, "Reinforcement learning for self-improving agent with skill library," *arXiv preprint*, 2025.

[34] C. Ling, P. Chen, A. Guan, J. Qu, S. A. Akbar, M. Gopinathan, and E. Cornejo, "PACE: Two-timescale self-evolution for small language model agents," *arXiv preprint*, 2026.

[35] P. Xia *et al.*, "SkillRL: Evolving agents via recursive skill-augmented reinforcement learning," *arXiv preprint*, 2026.

[36] X. Liu *et al.*, "SkillForge: Forging domain-specific, self-evolving agent skills in cloud technical support," *arXiv preprint*, 2026, accepted at ACM SIGIR 2026 Industry Track.

[37] X. Huang *et al.*, "CASCADE: Cumulative agentic skill creation through autonomous development and evolution," *arXiv preprint*, 2025.

[38] L. Gao, J. Schulman, and J. Hilton, "Scaling laws for reward model overoptimization," *arXiv preprint arXiv:2210.10760*, 2022.

[39] S. Shankar, J. Zamfirescu-Pereira, B. Hartmann, A. G. Parameswaran, and I. Arawjo, "Who validates the validators? aligning LLM-assisted evaluation of LLM outputs with human preferences," *arXiv preprint arXiv:2404.12272*, 2024.

[40] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, "The AI scientist: Towards fully automated open-ended scientific discovery," *arXiv preprint arXiv:2408.06292*, 2024.

[41] J. Xu *et al.*, "NanoResearch: Co-evolving skills, memory, and policy for personalized research automation," *arXiv preprint*, 2026.

[42] A. Qu, H. Zheng, Z. Zhou, Y. Yan, Y. Tang, S. Y. Ong, F. Hong, K. Zhou, C. Jiang, M. Kong, J. Zhu, X. Jiang, S. Li, C. Wu, B. K. H. Low, J. Zhao, and P. P. Liang, "CORAL: Towards autonomous multi-agent evolution for open-ended discovery," *arXiv preprint*, 2026.

[43] K. T.-i. Ong, M. Kang, D. Choi, J. Cho, S. Kim, S. Lim, G. Jang, M. Oh, B. Jeong, S. Kim, T. Kwon, and J. Yeo, "Towards direct evaluation of harness optimizers via priority ranking," *arXiv preprint*, 2026.

[44] Y. Hu, M. Xu, H. Guo, Y. Song, X. Zhu, and B. Zheng, "Skillchain: Closing the loop on skill evolution for image-based e-commerce ai assistants," *arXiv preprint arXiv:2606.12984*, 2026.

[45] J. Zhu, K. Mao, Y. Guo, D. He, S. Xu, S. Gu, and Y. Yue, "Skillcoach: Self-evolving rubrics for evaluating and enhancing agentic skill-use," *arXiv preprint arXiv:2607.01874*, 2026.

[46] A. Gunjal, A. Wang, E. Lau, V. Nath, Y. He, B. Liu, and S. Hendryx, "Rubrics as rewards: Reinforcement learning beyond verifiable domains," *arXiv preprint arXiv:2507.17746*, 2025.

[47] R. Jia, Y. Yang, Y. Gai, K. Luo, S. Huang, J. Lin, X. Jiang, and G. Jiang, "Writing-zero: Bridge the gap between non-verifiable tasks and verifiable rewards," *arXiv preprint arXiv:2506.00103*, 2025.

[48] K. Shi, Y. Liu, P. Wang, A. R. Fabbri, S. Joty, and A. Cohan, "References improve LLM alignment in non-verifiable domains," *arXiv preprint arXiv:2602.16802*, Feb. 2026.

[49] X. Jin, J. Zhang, T. Cao, and B. Yang, "Beyond score prediction: LLM-based essay scoring and feedback generation via reinforcement learning with rubric rewards," *arXiv preprint arXiv:2607.19219*, Jul. 2026.

[50] T. Yu, B. Ji, S. Wang, S. Yao, Z. Wang, G. Cui, L. Yuan, N. Ding, Y. Yao, Z. Liu, M. Sun, and T.-S. Chua, "RLPR: Extrapolating RLVR to general domains without verifiers," *arXiv preprint arXiv:2506.18254*, Jun. 2025.

[51] R. Xu, T. Liu, Z. Dong, T. Yu, I. Hong, C. Yang, L. Zhang, T. Zhao, and H. Wang, "Alternating reinforcement learning for rubric-based reward modeling in non-verifiable LLM post-training," *arXiv preprint arXiv:2602.01511*, 2026.

[52] H. Ding, B. Huang, Y. Fang, W. Liao, Z. Li, J. Zhang, Z. Wu, J. Zhao, and Y. Wang, "EvoRubrics: Dynamic rubrics as rewards via adversarial co-evolution for LLM reinforcement learning," *arXiv preprint arXiv:2606.23038*, Jun. 2026.

[53] H. Zhu, C. Cai, Y. Song, X. Chen, S. Han, and Y. Guo, "Self-evolving deep research via joint generation and evaluation," *arXiv preprint arXiv:2606.04507*, Jun. 2026.

[54] G. Li, B. Dalvi Mishra, Z. Wang, J. Yan, Y. Chen, C.-L. Li, L. T. Le, R. Han, G. Lee, H. Tong, C.-Y. Lee, and T. Pfister, "RubricEM: Meta-RL with rubric-guided policy decomposition beyond verifiable rewards," *arXiv preprint arXiv:2605.10899*, May 2026.

[55] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, "Cognitive architectures for language agents (CoALA)," *Transactions on Machine Learning Research (TMLR)*, 2023.

[56] M. Helwig, "Memory as infrastructure: Reliability engineering for persistent agent memory in months-long LLM-assisted development," *arXiv preprint arXiv:2609.05510*, Aug. 2026.

[57] A. Miyai, K. Aizawa, and T. Yamasaki, "Task-coevolve: Efficient harness optimization via adaptive validation task selection," *arXiv preprint arXiv:2608.20169*, 2026.

[58] S. Park, W. Kim, R. Tan, J. Zhang, W.-S. Han, P. Gao, C. Park, Y. Yao, R. Fu, E. Nallipogu, Q. Lin, S. Rajmohan, and D. Zhang, "Autosaddler: Automatic harness optimization with durable updates from agent execution traces," *arXiv preprint arXiv:2608.23041*, 2026.

[59] J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, and Y.-G. Jiang, "Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses," *arXiv preprint*, 2026.

[60] T. Zehle, "CANTANTE: Optimizing agentic systems via contrastive credit attribution," *arXiv preprint*, 2026.

[61] Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn, "Meta-Harness: End-to-end optimization of model harnesses," *arXiv preprint*, 2026.

[62] M. Nguyen, W. C. Tan, N. A. Hassan, A. Raman, L. H. Lim, and A. F. Razak, "Where does harness-optimization value live? Localized gains and the budget-splitting trap in self-evolving LLM agents," *arXiv preprint arXiv:2609.02889*, 2026.

[63] H. Liu, H. Yang, T. Jiang, B. Tang, F. Xiong, Y. Luo, and Z. Li, "SkillsVote: Lifecycle governance of agent skills from collection, recommendation to evolution," *arXiv preprint arXiv:2605.18401*, 2026.

[64] Y. Shen, B. Li, and X. Zhang, "SkillOpt-Lite: Better and faster agent self-evolution via one line of vibe," *arXiv preprint arXiv:2607.03451*, 2026.

[65] C. Shen and E. Hruschka, "Who maintains agent skills? A longitudinal study of human-governed, AI-assisted skill maintenance," *arXiv preprint arXiv:2609.05677*, 2026.

[66] X. Zhang, Y. Cui, G. Wang, Z. Li, W. Qiu, B. Zhu, and P. He, "Library drift: Diagnosing and fixing a silent failure mode in self-evolving llm skill libraries," *arXiv preprint arXiv:2605.19576*, 2026.

[67] Y. Wei, Z. Huang, S. Lu, J. Qian, Q. Wang, C. Wu, and L. He, "Skillsmith: Co-evolving skills and tools for self-improving agent systems," *arXiv preprint arXiv:2606.01314*, 2026.

[68] H. Pu, X. Song, and L. Zhao, "Skillops: Managing llm agent skill libraries as self-maintaining software ecosystems," *arXiv preprint arXiv:2605.13716*, 2026.

[69] Y. Wang, Y. Zhou, Y. Liang, C. Zhang, F. Liu, J. Zhou, and H. Yao, "Not all skills help: Measuring and repairing agent knowledge," *arXiv preprint arXiv:2606.15390*, 2026.

[70] X. Zhang, Y. Cui, G. Wang, Z. Li, W. Qiu, B. Zhu, and P. He, "Ratchet: A minimal hygiene recipe for self-evolving LLM agents," *arXiv preprint*, 2026.

[71] X. Wu *et al.*, "Co-evolving LLM decision and skill bank agents for long-horizon tasks," *arXiv preprint*, 2026.

[72] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber, "Language agents as optimizable graphs," in *Forty-first International Conference on Machine Learning (ICML 2024)*, 2024, arXiv:2402.16823.

[73] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu, "AFlow: Automating agentic workflow generation," *arXiv preprint arXiv:2410.10762*, 2024.

[74] J. Chu, J. Seo, J. Cho, J. Na, Y. Xiong, Y. Kim, and H. J. Kim, "Agentgrad: Intervention-guided prompt optimization for multi agent systems," *arXiv preprint arXiv:2609.08572*, 2026.

[75] M. Chen, C. Lv, G. Zhang, H. Chang, and S. Zhou, "Harnessforge: Joint harness and policy evolution for adaptive agent systems," *arXiv preprint arXiv:2606.01779*, 2026.

[76] S. Zhao, R. Kong, Z. Zhang, L. Shen, X. He, X. Zhang, and Q. Zhang, "Topological coherence for self-evolving multi-agent systems," *arXiv preprint arXiv:2609.37953*, 2026.

[77] Y. Cheng, C. Liu, H. Yu, H. Tan, T. Wu, W. Jin, J. Ran, K. Wang, X. Li, H. Bai, G. Chesi, and N. Wong, "Optimas: Automatically optimize multi-agent system," *arXiv preprint arXiv:2608.21918*, 2026.

[78] AGENTS.md, "AGENTS.md: A cross-vendor open standard for agent instructions," 2025. [Online]. Available: https://agents.md

[79] R. Robbes, T. Matricon, T. Degueule, A. Hora, and S. Zacchiroli, "Agentic much? adoption of coding agents on github," *arXiv preprint arXiv:2601.18341*, 2026.

[80] M. Galster, S. Mohsenimofidi, J. L. Lulla, M. A. Abubakar, C. Treude, and S. Baltes, "Harness engineering for agentic ai coding tools: An exploratory study," *arXiv preprint arXiv:2602.14690*, 2026.

[81] T. Gloaguen, N. Mündler-Sasahara, M. N. Müller, V. Raychev, and M. Vechev, "Evaluating agents.md: Are repository-level context files helpful for coding agents?" *arXiv preprint arXiv:2602.11988*, 2026.

[82] J. L. Lulla *et al.*, "On the impact of AGENTS.md files on the efficiency of AI coding agents," *arXiv preprint*, 2026.

[83] T. Q. Nguyen, S. Desai, R. H. Anwar, F. Shaik, V. Suryanarayanan, and V. Chowdhary, "VerificAgent: Domain-specific memory verification for scalable oversight of aligned computer-use agents," *arXiv preprint arXiv:2506.02539*, Jun. 2025.

[84] Y. Wang, L. Xia, R. Liu, Z. Zhang, H. Wu, Y. Yang, J. He, Y. Guo, and K. Lei, "Sage: A statistical acceptance gate for self-evolving agents," *arXiv preprint arXiv:2609.36043*, 2026.

[85] L. Shang, M. Xu, Y. Sun, T. Xia, L. Hu, L. Xu, and N. Zheng, "When self-evolution backfires: Pre-commit gating against skill contamination in llm agents," *arXiv preprint arXiv:2608.05810*, 2026.

[86] Letta, "MemFS; memory & dreaming; introducing context repositories: Git-based memory for coding agents," https://docs.letta.com/letta-code/memfs, https://docs.letta.com/letta-agent/memory, https://www.letta.com/blog/context-repositories/, 2026.

[87] Mem0, "Mem0 platform: Memory history, feedback, and changelog," https://docs.mem0.ai/api-reference/memory/history-memory, https://docs.mem0.ai/api-reference/memory/feedback, https://docs.mem0.ai/changelog, 2026.

[88] Zep, "Zep documentation: Facts; february 2026 deprecation wave; changelog," https://help.getzep.com/facts, https://help.getzep.com/february-2026-deprecation-wave, https://help.getzep.com/changelog/2026/9/24, 2026.

[89] Z. Wang, "TOKI: A bitemporal operator algebra for contradiction resolution in LLM-agent persistent memory," *arXiv preprint arXiv:2606.06240*, Jun. 2026.

[90] J. Xu, Y. Xiao, W. Shao, H. Liu, and X. Li, "Memory provenance laundering in LLM agents: A non-amplification firewall for persistent memory," *arXiv preprint arXiv:2607.29167*, Jul. 2026.

[91] Y. Margalit, N. Cohen-Inger, E. Avram, R. Taig, and O. Margalit, "Governed shared memory for multi-agent LLM systems," *arXiv preprint arXiv:2606.24535*, Jun. 2026.

[92] X. Sun, Q. Wang, H. Li, and L. Chen, "Verifiable memory: Learning unified memory management with local and global verifiers for large language model agents," *arXiv preprint arXiv:2608.03137*, Aug. 2026.

[93] Z. Yu, Y. Wu, Z. Yin, K. Chen, Z. Zhao, M. Wang, S. Yan, and L. Yang, "Recursive experiential–working memory evolution for long-horizon agent harnesses," *arXiv preprint arXiv:2608.24876*, Aug. 2026.

[94] H. Wang, G. Dong, H. Liang, Z. Zhang, J. Luo, C. Liu, C. Xue, and H. Tang, "MemGuard: Persisting verifier signals for LLM-agent memory governance," *arXiv preprint arXiv:2608.21867*, Aug. 2026.

[95] D. F. Cuadros, A.-A. Maiga, H. Meskhidze, and A. Curtis-Trudel, "Governed collaborative memory as artificial selection in LLM-based multi-agent systems," *arXiv preprint*, 2026.

[96] Y. T. Shen, K. Toyoda, and A. Leung, "Revoked but still authoritative: An empirical study of revocation enforcement in agent-memory systems," *arXiv preprint arXiv:2609.08258*, Sep. 2026.

[97] H. Gao, H. Chen, C. Wang, S. Guo, L. Pang, Z. Liu, H. Shen, and X. Cheng, "Skillaudit: Ground-truth-free skill evolution via paired trajectory auditing," *arXiv preprint arXiv:2606.14239*, 2026.

[98] T. Sah, D. Sah, H. Jain, and T. Sah, "Evoundo: Recoverability-constrained self-evolution for llm agent harnesses," *arXiv preprint arXiv:2608.28363*, 2026.

[99] J. Hu, H. Zhao, Q. Jia, D. Chen, T. Zhao, S. Duan, and G. Liu, "Learning from others, acting for you: Cross-user memory sharing for llm agents," *arXiv preprint arXiv:2609.32511*, 2026.

[100] Y. Guo, H. Zhou, J. Zhang, Y. Zhang, Y. Liu, Q. Chen, Q. Lin, H. Dong, J. M. Zhang, and Z. Chen, "Xreposkill: Learning transferable skills for software engineering agents," *arXiv preprint arXiv:2609.36807*, 2026.

[101] R. Wang and J. Cui, "Evo2team: When do evolved skills transfer? from selection to deployment," *arXiv preprint arXiv:2609.34135*, 2026.

[102] J. Nie, Y. Zhang, Q. Cai, Y.-m. Cheung, X. Tian, and B. Han, "Evolvenet: Collaborative harness evolution for agent self-improvement," *arXiv preprint arXiv:2608.04968*, 2026.

[103] J. Yang, G. Yao, Y. Zhang, R. R. Kompella, G. Liu, and S. Chang, "FederatedSkill: Federated learning for agentic skill evolution," *arXiv preprint arXiv:2606.03143*, 2026.

[104] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems (ADAS)," *arXiv preprint*, 2024.

[105] G. Wang *et al.*, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint*, 2023.

[106] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, pp. 468–475, 2024.

[107] Sierra, "Agent OS 2.0: From answers to memory and action," 2025. [Online]. Available: https://sierra.ai/blog/agent-os-2-0

[108] X. Wang *et al.*, "AutoAgent: Evolving cognition and elastic memory orchestration for adaptive agents," *arXiv preprint*, 2026.

[109] P. Barbaste, T. Darrigol, G. Vu, and T. Wiltberger, "Harness engineering: Anatomy, architecture, and evolution of coding agents – a source-code study of eleven systems," *arXiv preprint arXiv:2609.00006*, 2026.

[110] J. L. Lulla, V. Nersesyan, S. Mohsenimofidi, C. Treude, and S. Baltes, "Loop engineering: Building blocks, adoption, and impact," *arXiv preprint arXiv:2608.21884*, 2026.

[111] agentic-community, "Mcp gateway & registry," https://github.com/agentic-community/mcp-gateway-registry, 2026.

[112] Microsoft, "Inventory and discover mcp servers in your api center," https://learn.microsoft.com/en-us/azure/api-center/register-discover-mcp-server, 2026.

[113] TrueFoundry, "Mcp gateway authentication and security," https://www.truefoundry.com/docs/ai-gateway/mcp-gateway-auth-security, 2026.

[114] R. Kang and V. Wang, "Registry-governed agent lifecycle: Completing EDDOps with evaluation-driven registration, promotion, and retirement on AWS AgentCore," *arXiv preprint arXiv:2607.00345*, Jul. 2026.

[115] A. Quessada-Vial, "Agentic configuration management (ACM): A reference configuration model for governed agentic systems," *arXiv preprint arXiv:2608.11166*, Aug. 2026.

[116] X. Qin, S. Luan, J. See, Z. Boukhers, C. Yang, and Z. Li, "Governed capability evolution: Lifecycle-time compatibility checking and rollback for AI-component-based systems, with a proof-of-concept evaluation on embodied agents," *arXiv preprint arXiv:2604.08059*, Apr. 2026.

[117] Braintrust, "Use prompts in code," Braintrust documentation, https://www.braintrust.dev/docs/guides/prompts, 2026, accessed 2026-10-03.

[118] Vellum, "Release tags," Vellum documentation, https://docs.vellum.ai/product/deployments/release-tags, 2026, accessed 2026-10-03.

[119] LangChain, "Prompt engineering concepts," LangSmith documentation, https://docs.langchain.com/langsmith/prompt-engineering-concepts, 2026, accessed 2026-10-03.

[120] Z. Jiang and B. Dong, "Buildrix: An open platform for sharing and benchmarking agentic ai skills in building engineering," *arXiv preprint arXiv:2606.25139*, 2026.

[121] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab, "GEPA: Reflective prompt evolution can outperform reinforcement learning," *arXiv preprint*, 2025.

[122] N. Vassilyev, W. Berrios, R. Zhang, B. Han, D. Kiela, and S. Mehri, "Reflective context learning: Studying the optimization primitives of context space," *arXiv preprint*, 2026.

[123] Q. Cai, Y. Zhang, X. Jia, H. Zheng, W. Xue, J. Song, X. Tian, and Y. Guo, "MOSS: Self-evolution through source-level rewriting in autonomous agent systems," *arXiv preprint*, 2026.

[124] L. Li, J. Li, J. Zhao, K. Ye, Y. Li, C.-z. Xu, and X. Gao, "Grow the harness, not the context: From strategy-free scaffolds to reusable specialist agents," *arXiv preprint arXiv:2609.26760*, 2026.

[125] M. M. Tanjim, J. Subramanian, X. Chen, B. Kveton, S. Mukherjee, A. Zhang, S. Kim, S. Sarkhel, and S. Choudhury, "MOCHA: Multi-objective chebyshev annealing for agent skill optimization," *arXiv preprint*, 2026.

[126] Y. Shalev, Z. Ding, and M. Jamnik, "Training language agents to learn from experience," *arXiv preprint*, 2026.

[127] H. Li, R. He, Q. Zhang, C. Ji, Q. Mang, X. Chen, L. A. Agrawal, W.-L. Liao, E. Yang, A. Cheung, J. Zou, K. Olukotun, I. Stoica, and J. E. Gonzalez, "Combee: Scaling prompt learning for self-improving language model agents," *arXiv preprint*, 2026.

[128] P. Hebbar, Y. Manawat, S. Verboomen, A. Ivanova, S. Palanimalai, K. Bhatia, and V. Baskaran, "Sia: Self improving ai with harness & weight updates," *arXiv preprint arXiv:2605.27276*, 2026.

[129] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 54, 2017, pp. 1273–1282.

[130] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *International Conference on Learning Representations (ICLR)*, 2018, arXiv:1710.06963.

[131] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis *et al.*, "Advances and open problems in federated learning," *arXiv preprint arXiv:1912.04977*, 2019, published in Foundations and Trends in Machine Learning, vol. 14, no. 1–2, 2021.

[132] T. Guo, S. Guo, J. Wang, and W. Xu, "PromptFL: Let federated participants cooperatively learn prompts instead of models – federated learning in age of foundation model," *arXiv preprint arXiv:2208.11625*, 2022.

[133] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, 2015.

[134] D. Jaroslawicz *et al.*, "How many instructions can LLMs follow at once?" *arXiv preprint*, 2025.

[135] W. Zou *et al.*, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," *arXiv preprint*, 2024, accepted at USENIX Security 2025.

[136] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases," *arXiv preprint arXiv:2407.12784*, 2024.

[137] Z. Xu *et al.*, "From storage to steering: Memory control flow attacks on LLM agents," *arXiv preprint*, 2026.

[138] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," *arXiv preprint arXiv:2302.12173*, 2023.

[139] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," *arXiv preprint arXiv:2406.13352*, 2024.

[140] Check Point Research, "MCPoison: Cursor IDE's MCP vulnerability (CVE-2025-54136)," https://research.checkpoint.com/2025/cursor-vulnerability-mcpoison/, 2025.

[141] Y. Su, W. Xu, C. Zuo, and E. Bertino, "ChronoMem: Version control and semantic rollback for large language model agent memory," *arXiv preprint arXiv:2607.27773*, 2026.

[142] C. Yu, Y. Wang, J. Zhang, Y. Duan, M. Zheng, Z. Wu, K. Shi, and T. Cai, "From faulty memories to corrected actions: Dependency-guided rollback repair for memory-augmented agents," *arXiv preprint arXiv:2608.10502*, 2026.

[143] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," *arXiv preprint arXiv:2404.13208*, 2024.

[144] S. Singh *et al.*, "The leaderboard illusion," *arXiv preprint*, 2025.

[145] M. Cemri *et al.*, "Why do multi-agent LLM systems fail?" *arXiv preprint*, 2025.

[146] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics (TACL)*, 2023.

[147] Q. Yang, Z. Kou, X. Zhang, D.-D. Wu, H. Gu, J. Guo, Y. Liu, D. Jiang, and Q. Xu, "Federated agent optimization," *arXiv preprint arXiv:2610.01195*, 2026.

[148] H. Robbins and D. Siegmund, "A convergence theorem for non negative almost supermartingales and some applications," in *Optimizing Methods in Statistics*. Academic Press, 1971, pp. 233–257.

MIKHAIL L. ARBUZOV TODO(bio).

LEE MOSBACKER TODO(bio).

SISONG BEI TODO(bio).

ZIWEI DONG TODO(bio).

DMITRI KALAEV TODO(bio).

ALEXEY SHVETS TODO(bio).